

JADAVPUR UNIVERSITY  
DEPARTMENT OF PHYSICS

---

Numerical Solutions using the  
Finite Difference Method

---

*The Discretization Scheme*

$$\frac{d^2\psi_i}{dx^2} \approx \frac{\psi_{i+1} - 2\psi_i + \psi_{i-1}}{\Delta x^2}$$

**Submitted by:**  
Mohak Das

**Supervisor:**  
Dr. Asim Kumar Ghosh

**Class:** PG-I  
**Class Roll No:** 002520702028  
**Exam Roll No:** PPHD0261034

COMPUTATIONAL PHYSICS LAB

## Abstract

This report presents a comprehensive numerical investigation of the 1D Time-Independent Schrödinger Equation (TISE) utilizing the Finite Difference Method (FDM). The continuous differential equation was discretized into a symmetric tridiagonal matrix eigenvalue problem and solved efficiently using the TQLI (Tridiagonal QL Implicit) algorithm implemented in Fortran. A diverse suite of fourteen distinct quantum potentials was analyzed, ranging from foundational models—such as the infinite square well and harmonic oscillator—to complex anharmonic, asymmetric, and coupled-well systems. Furthermore, the computational methodology was extended to advanced physical applications, successfully modeling molecular dissociation via the Morse potential and singular boundary behavior via the Dirac-delta potential. Data visualization, handled systematically in Python, shifted the computed wavefunctions by their respective discrete eigen-energies. This approach provided clear visual confirmations of core quantum mechanical phenomena, including parity alternation, quantum penetration into classically forbidden regions, tunneling-induced spontaneous symmetry breaking, and the strict numerical distinction between bound and scattering states.

# Contents

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                          | <b>1</b>  |
| 1.1      | Main Idea . . . . .                                          | 1         |
| 1.2      | Derivation and Discretization . . . . .                      | 1         |
| 1.3      | Matrix Formulation . . . . .                                 | 1         |
| <b>2</b> | <b>Numerical Implementation</b>                              | <b>2</b>  |
| 2.1      | Global Constants and Precision . . . . .                     | 2         |
| 2.2      | The General FDM Solver . . . . .                             | 2         |
| 2.3      | Data Visualization (Python Setup) . . . . .                  | 4         |
| <b>3</b> | <b>Solution to the Assignment Problems</b>                   | <b>5</b>  |
| 3.1      | Problem 1: Slanted Base Infinite Potential Well . . . . .    | 5         |
| 3.2      | Problem 2: One-dimensional Half-Harmonic Potential . . . . . | 8         |
| 3.3      | Problem 3: One-dimensional Harmonic Potential . . . . .      | 11        |
| 3.4      | Problem 4: Finite Potential Well . . . . .                   | 14        |
| 3.5      | Problem 5: Parabolic Base Infinite Well . . . . .            | 17        |
| 3.6      | Problem 6: Modulo $x$ Potential . . . . .                    | 20        |
| 3.7      | Problem 7: Modulo $x^3$ Potential . . . . .                  | 23        |
| 3.8      | Problem 8: Quartic Potential . . . . .                       | 26        |
| 3.9      | Problem 9: Quadratic-Quartic Potential . . . . .             | 29        |
| 3.10     | Problem 10: Half-Cubic Potential . . . . .                   | 32        |
| 3.11     | Problem 11: Half-Quartic Potential . . . . .                 | 35        |
| <b>4</b> | <b>Additional Exercises</b>                                  | <b>38</b> |
| 4.1      | Problem 12: Double Well Potential . . . . .                  | 38        |
| 4.2      | Problem 13: The Morse Potential . . . . .                    | 41        |
| 4.3      | Problem 14: The Dirac Delta Potential . . . . .              | 44        |
| <b>5</b> | <b>Remarks</b>                                               | <b>47</b> |

# 1 Introduction

## 1.1 Main Idea

Analytically solving the 1D Time-Independent Schrödinger Equation (TISE) is only possible for a handful of idealized potentials, such as the infinite square well or the simple harmonic oscillator. For arbitrary or complex potentials, exact analytical solutions do not exist. To study these systems, we must rely on numerical approximations. The Finite Difference Method (FDM) provides a robust framework for this by converting the continuous differential equation into a discrete algebraic eigenvalue problem that computers can solve efficiently. It is particularly powerful because it can handle any potential shape simply by evaluating the potential function at discrete spatial points.

## 1.2 Derivation and Discretization

We begin with the 1D Time-Independent Schrödinger Equation:

$$-\frac{\hbar^2}{2m} \frac{d^2\psi(x)}{dx^2} + V(x)\psi(x) = E\psi(x)$$

To discretize this, we divide the spatial domain into  $N$  equally spaced grid points with a step size of  $\Delta x$ . Let  $x_i$  be the position of the  $i$ -th grid point, and  $\psi_i = \psi(x_i)$  be the wavefunction value at that point. Using the central difference approximation for the second derivative, we get:

$$\frac{d^2\psi_i}{dx^2} \approx \frac{\psi_{i+1} - 2\psi_i + \psi_{i-1}}{\Delta x^2}$$

Substituting this back into the TISE yields the discrete equation for each grid point  $i$ :

$$-\frac{\hbar^2}{2m\Delta x^2}(\psi_{i+1} - 2\psi_i + \psi_{i-1}) + V(x_i)\psi_i = E\psi_i$$

## 1.3 Matrix Formulation

Grouping the terms by  $\psi_{i-1}$ ,  $\psi_i$ , and  $\psi_{i+1}$ , we can rewrite the equation as:

$$\left( \frac{\hbar^2}{2m\Delta x^2} + V(x_i) \right) \psi_i - \frac{\hbar^2}{2m\Delta x^2} \psi_{i+1} - \frac{\hbar^2}{2m\Delta x^2} \psi_{i-1} = E\psi_i$$

This forms a system of linear equations. Considering  $\psi_0 = \psi_{N+1} = 0$  (Dirichlet Boundary conditions), the above set of linear equations can be written in a matrix equation form:

$$\begin{pmatrix} 2K + V(x_1) & -K & 0 & \cdots & 0 \\ -K & 2K + V(x_2) & -K & \cdots & 0 \\ 0 & -K & 2K + V(x_3) & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 2K + V(x_N) \end{pmatrix} \begin{pmatrix} \psi_1 \\ \psi_2 \\ \psi_3 \\ \vdots \\ \psi_N \end{pmatrix} = E \begin{pmatrix} \psi_1 \\ \psi_2 \\ \psi_3 \\ \vdots \\ \psi_N \end{pmatrix}$$

where  $K = \frac{\hbar^2}{2m\Delta x^2}$

- Main Diagonal Elements ( $d_i$ ):  $d_i = \frac{\hbar^2}{m\Delta x^2} + V(x_i)$
- Off-Diagonal Elements ( $e_i$ ):  $e_i = -\frac{\hbar^2}{2m\Delta x^2}$

## 2 Numerical Implementation

### 2.1 Global Constants and Precision

To ensure numerical stability and physical consistency across all problems, a shared module `constants_mod` is used. This module defines the double-precision floating-point kind (`real64`) and sets the atomic units ( $\hbar = 1, m = 1$ ).

```

1 module constants_mod
2   implicit none
3   integer, parameter :: dp = selected_real_kind(15, 307)
4   real(dp), parameter :: h_bar = 1.0_dp
5   real(dp), parameter :: mass = 1.0_dp
6   real(dp), parameter :: pi = 3.1415926535897932384626433832795_dp
7 contains
8 end module constants_mod

```

Listing 1: Global Constants Module

### 2.2 The General FDM Solver

The core logic is encapsulated in the `FDM_toolbox` module. To diagonalize the Hamiltonian matrix, the TQLI (Tridiagonal QL Implicit) algorithm was implemented. Standard eigenvalue solvers (like the Jacobi method) require  $O(N^3)$  operations, which becomes computationally prohibitive for large grids. TQLI exploits the tridiagonal nature of our specific matrix, finding eigenvalues in highly efficient  $O(N^2)$  time using Givens rotations and Wilkinson shifts to accelerate convergence. To organize the results physically, a custom `sort_eigen` subroutine was used to arrange the eigenvalues in ascending order ( $E_0 < E_1 < E_2 \dots$ ), simultaneously rearranging the corresponding eigenvector columns to maintain the correct energy-state pairings.

```

1 module FDM_toolbox
2   use constants_mod
3   implicit none
4
5 contains
6   subroutine tqli(n, d, e, z)
7     integer, intent(in) :: n
8     real(dp), intent(inout) :: d(n), e(n), z(n,n)
9
10    integer :: m, l, iter, i, k
11    real(dp) :: s, c, p, g, r, f, b, dd
12
13    ! Renumbering the e(n)
14    do i = 2, n
15      e(i-1) = e(i)
16    end do
17    e(n) = 0.0_dp
18

```

```

19     do l = 1, n
20         iter = 0
21         do
22             ! checking if the diagonal element is the eigenvalue
23             do m = l, n-1
24                 dd = abs(d(m)) + abs(d(m+1))
25                 if (abs(e(m)) + dd == dd) exit
26             end do
27
28             if (m == l) exit
29
30             if (iter == 30) then
31                 write(*, '(A)') "Too many iterations in tqli"
32                 stop
33             end if
34             iter = iter + 1
35
36             ! Wilkinson shift to accelerate convergence
37             g = (d(l+1) - d(l))/(2.0_dp * e(l))
38             r = sqrt(g**2 + 1.0_dp**2)
39
40             ! g = d(m) - d(l) + e(l) / (g + sign(r))
41             if (g >= 0) then
42                 g = d(m) - d(l) + e(l) / (g + r)
43             else
44                 g = d(m) - d(l) + e(l) / (g - r)
45             end if
46
47             ! Givens Rotation
48             s = 1.0_dp
49             c = 1.0_dp
50             p = 0.0_dp
51
52             do i = m-1, l, -1
53                 f = s*e(i)
54                 b = c*e(i)
55                 r = sqrt(f**2 + g**2)
56                 e(i+1) = r
57
58                 if ( r == 0.0_dp ) then
59                     d(i+1) = d(i+1) - p
60                     e(m) = 0.0_dp
61                     exit
62                 end if
63
64                 s = f/r
65                 c = g/r
66                 g = d(i+1) - p
67                 r = (d(i) - g) * s + 2.0_dp * c * b
68                 p = s*r
69                 d(i+1) = g + p
70                 g = c*r-b
71
72                 ! Accumulate rotations into Eigenvectors Z
73                 do k = 1, n
74                     f = z(k, i+1)

```

```

75         z(k, i+1) = s*z(k, i) + c*f
76         z(k, i) = c*z(k, i) - s*f
77     end do
78 end do
79 d(1) = d(1) - p
80 e(1) = g
81 e(m) = 0.0_dp
82 end do
83 end do
84 end subroutine tqli
85
86 subroutine sort_eigen(n, d, z)
87     integer, intent(in) :: n
88     real(dp), intent(inout) :: d(n), z(n, n)
89
90     integer :: i, j, k
91     real(dp) :: temp
92
93     do i = 1, n-1
94         k = i
95         temp = d(i)
96         do j = i+1, n
97             if ( d(j) < temp ) then
98                 k = j
99                 temp = d(j)
100             end if
101         end do
102         if ( k /= i ) then
103             d(k) = d(i)
104             d(i) = temp
105             do j = 1, n
106                 temp = z(j, i)
107                 z(j, i) = z(j, k)
108                 z(j, k) = temp
109             end do
110         end if
111     end do
112     do i = 1, n
113         k = maxloc(abs(z(:, i))), 1)
114         if ( z(k, i) < 0.0_dp ) then
115             z(:, i) = -1.0_dp * z(:, i)
116         end if
117     end do
118 end subroutine sort_eigen
119 end module FDM_toolbox

```

Listing 2: FDM Toolbox Implementation

## 2.3 Data Visualization (Python Setup)

To clearly visualize the quantum states and their classical turning points, the wavefunctions were scaled by a visual factor and shifted by their corresponding energy eigenvalues ( $E_n$ ). The following generalized Python script serves as the core visualization template for all problems. For each specific potential, placeholders such as the input file, energy list, and scale factor were dynamically adjusted.

```

1      import numpy as np
2      import matplotlib.pyplot as plt
3
4      # --- Configuration Placeholders ---
5      DATA = np.loadtxt("data/<FILENAME>.dat")
6      ENERGIES = np.loadtxt("data/<FILENAME>_energies.dat") #
Eigenvalues from Fortran solver
7      SCALE_FACTOR = <VALUE> # Visual scaling to prevent wavefunction
overlap
8
9      # --- Data Loading ---
10     # Columns: x, V(x), psi_0, psi_1, psi_2, ...
11     x = DATA[:, 0]
12     V_x = DATA[:, 1]
13
14     # Extract wavefunctions dynamically based on the number of
energies provided
15     wavefunctions = [DATA[:, i] for i in range(2, len(ENERGIES) + 2)]
16
17     # --- Visualization ---
18     plt.figure(figsize=(8, 8))
19
20     # Plot the background potential
21     plt.plot(x, V_x, color='black', linewidth=2, label='V(x)')
22
23     # Plot shifted and scaled wavefunctions
24     for n, (psi, E) in enumerate(zip(wavefunctions, ENERGIES)):
25         plt.plot(x, (psi * SCALE_FACTOR) + E, label=f'n={n} (E={E:.4f
})')
26         plt.axhline(E, color='gray', linestyle='--', alpha=0.5) #
Energy level guide
27
28     # Formatting
29     plt.title("<PROBLEM_TITLE>", size=15)
30     plt.xlabel(r'Position $(x)$', size=15)
31     plt.ylabel(r'Wavefunction $\psi(x)$', size=15)
32     plt.xlim(<X_MIN>, <X_MAX>)
33     plt.ylim(<Y_MIN>, <Y_MAX>)
34
35     # Push legend outside the right edge of the plot
36     plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.15), ncol
=3)
37     plt.subplots_adjust(bottom=0.2)
38     plt.tight_layout()
39     plt.show()
40

```

## 3 Solution to the Assignment Problems

### 3.1 Problem 1: Slanted Base Infinite Potential Well

- **Description:** An infinite well where the floor is not flat, but slopes upwards linearly.
- **Potential:**  $V(x) = \frac{V_0}{L}x$  for  $0 \leq x \leq L$ , and  $\infty$  elsewhere.  $V_0 = 200$ ,  $L = 1$ .

• Code:

```

1  ! =====
2  ! File    : prob01_infinite_potential_well_slanted_base .f90
3  ! Author  : Mohak Das
4  ! Course  : Computational Lab: Quantum Mechanics
5  ! Topic   : FDM approximation for Infinite Square Well With Slanted
6              Base
7  ! Units   :  $m = \hbar = 1$ 
8  ! Method  : Finite Difference Method
9  ! =====
10
11 module prob01_physics
12     use constants_mod
13     implicit none
14     real(dp), parameter :: V0 = 200.0_dp
15     integer, parameter :: N = 2001
16     real(dp) :: L_well = 1.0_dp
17     real(dp) :: L_box = 1.0_dp
18     real(dp) :: x, dx
19     real(dp) :: d(N), e(N), z(N, N)
20     integer :: i
21
22 contains
23     subroutine initialize()
24         dx = L_box / real(N + 1, dp)
25         z = 0.0_dp
26
27         do i = 1, N
28             z(i, i) = 1.0_dp
29         end do
30
31         do i = 1, N
32             x = real(i, dp) * dx
33             d(i) = (( $\hbar^2$ ) / (mass * dx**2)) + potential(x)
34             if ( i < N ) then
35                 e(i) = - ( $\hbar^2$ ) / (2 * mass * dx**2)
36             else
37                 e(N) = 0.0_dp
38             end if
39         end do
40     end subroutine initialize
41
42     function potential(x_vals) result(y)
43         real(dp), intent(in) :: x_vals
44         real(dp) :: y
45
46         y = V0/L_well * x_vals
47     end function potential
48 end module prob01_physics
49
50 program prob01_driver
51     use FDM_toolbox
52     use prob01_physics
53     implicit none
54

```

```

55  call initialize()
56  call tqli(N, d, e, z)
57  call sort_eigen(N, d, z)
58
59  write(*, '(A)') "
-----"
60  write(*, '(A)') "Problem 1: Infinite Potential Well With Slanted
Base"
61  write(*, '(A)') "
-----"
62  write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar
= ", h_bar
63  write(*, '(A)') "Method: Finite Difference Method"
64  write(*, '(A)') "
-----"
65  write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'
) " "
66
67  open(unit=10, file='data/prob01_energies.dat', status='replace')
68  do i = 1, 5
69      write(*, '(A, I2, A, G0.10)') "n = ", i, "      E = ", d(i)
70      write(10, *) d(i)
71  end do
72  close(10)
73
74  open(unit=10, file='data/prob01.dat', status='replace')
75  do i = 1, N
76      x = real(i, dp) * dx
77      write(10, *) x, potential(x), z(i,1), z(i,2), z(i,3), z(i, 4)
, z(i, 5)
78  end do
79  close(10)
80  print *, "Wavefunctions written to prob01.dat"
81 end program prob01_driver

```

- **Output:**

```

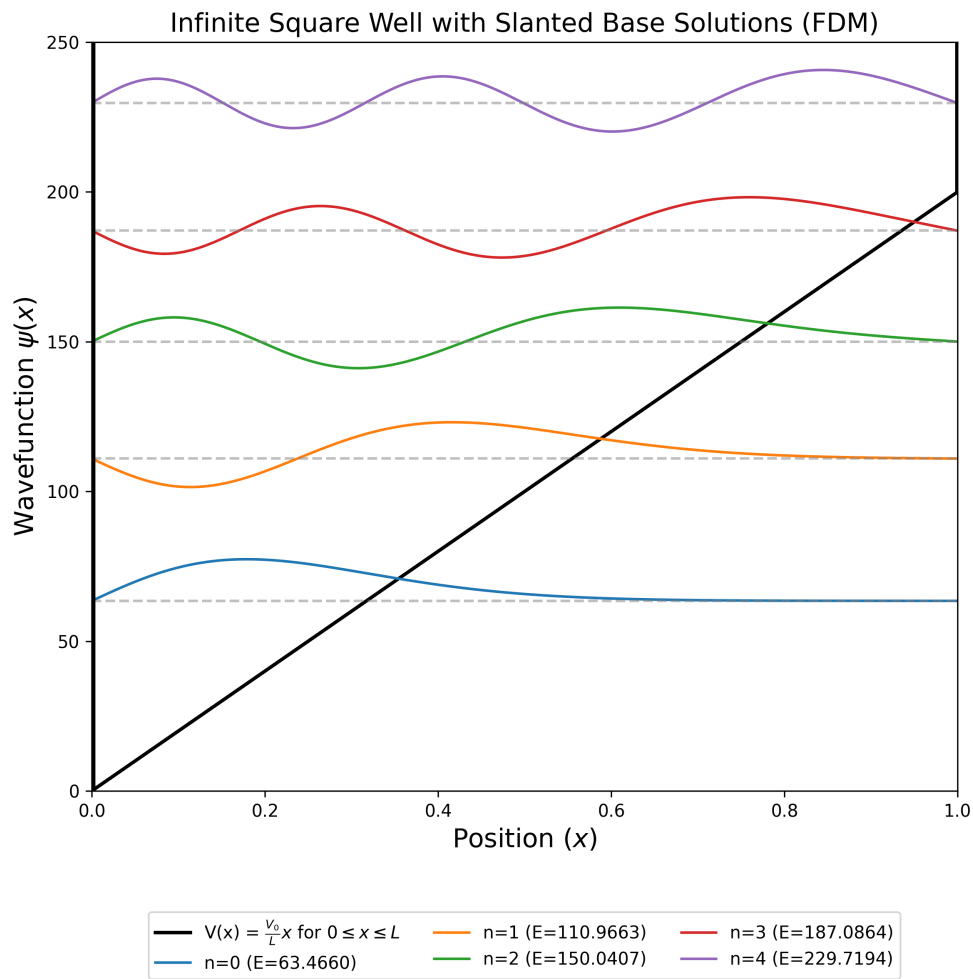
-----
Problem 1: Infinite Potential Well With Slanted Base
-----
Unit used: m = 1.0, h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n = 1      E = 63.46596852
n = 2      E = 110.9662942
n = 3      E = 150.0406729
n = 4      E = 187.0863589
n = 5      E = 229.7193690
Wavefunctions written to prob01.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 300.0 to prevent overlap between the

energy levels.



- **Plot Analysis:** Unlike the standard square well, the wavefunctions are pushed towards the lower-potential side (left side). The symmetry is broken, and the expectation value of position  $\langle x \rangle$  shifts toward  $x = 0$ .

### 3.2 Problem 2: One-dimensional Half-Harmonic Potential

- **Description:** A harmonic oscillator cut in half by an infinite wall at the origin.
- **Potential:**  $V(x) = \frac{1}{2}m\omega^2x^2$  for  $x \geq 0$ , and  $\infty$  otherwise.
- **Code:**

```

1 !=====
2 ! File      : prob02_half_harmonic.f90
3 ! Author    : Mohak Das
4 ! Course    : Computational Lab: Quantum Mechanics
5 ! Topic     : FDM approximation for Half Harmonic Oscillator
6 ! Units     : m = h_bar = 1
7 ! Method    : Finite Difference Method
8 !=====

```

```

9
10
11 module prob02_physics
12     use constants_mod
13     implicit none
14     real(dp), parameter :: omega = 1.0_dp
15     integer, parameter :: N = 2001
16     real(dp) :: L = 10.0_dp
17     real(dp) :: x, dx
18     real(dp) :: d(N), e(N), z(N, N)
19     integer :: i
20
21 contains
22     subroutine initialize()
23         dx = L / real(N + 1, dp)
24         z = 0.0_dp
25
26         do i = 1, N
27             z(i, i) = 1.0_dp
28         end do
29
30         do i = 1, N
31             x = real(i, dp) * dx
32             d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x)
33             if ( i < N ) then
34                 e(i) = - (h_bar**2) / (2 * mass * dx**2)
35             else
36                 e(N) = 0.0_dp
37             end if
38         end do
39     end subroutine initialize
40
41     function potential(x_vals) result(y)
42         real(dp), intent(in) :: x_vals
43         real(dp) :: y
44
45         y = 0.5_dp * mass * (omega**2) * (x_vals**2)
46     end function potential
47 end module prob02_physics
48
49 program prob02_driver
50     use FDM_toolbox
51     use prob02_physics
52     implicit none
53
54     call initialize()
55     call tqli(N, d, e, z)
56     call sort_eigen(N, d, z)
57
58     write(*, '(A)') "-----"
59     write(*, '(A)') "Problem 2: Half Harmonic Oscillator"
60     write(*, '(A)') "-----"
61
62     write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar
63     = ", h_bar

```

```

62     write(*, '(A)') "Method: Finite Difference Method"
63     write(*, '(A)') "-----"
64     write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'
65     ) " "
66
67     open(unit=10, file='data/prob02_energies.dat', status='replace')
68     do i = 1, 5
69         write(*, '(A, I2, A, G0.10)') "n = ", i, "      E = ", d(i)
70         write(10, *) d(i)
71     end do
72     close(10)
73
74     open(unit=10, file='data/prob02.dat', status='replace')
75     do i = 1, N
76         x = real(i, dp) * dx
77         write(10, *) x, potential(x), z(i,1), z(i,2), z(i,3), z(i, 4)
78         , z(i, 5)
79     end do
80     close(10)
81     print *, "Wavefunctions written to prob02.dat"
82 end program prob02_driver

```

### • Output:

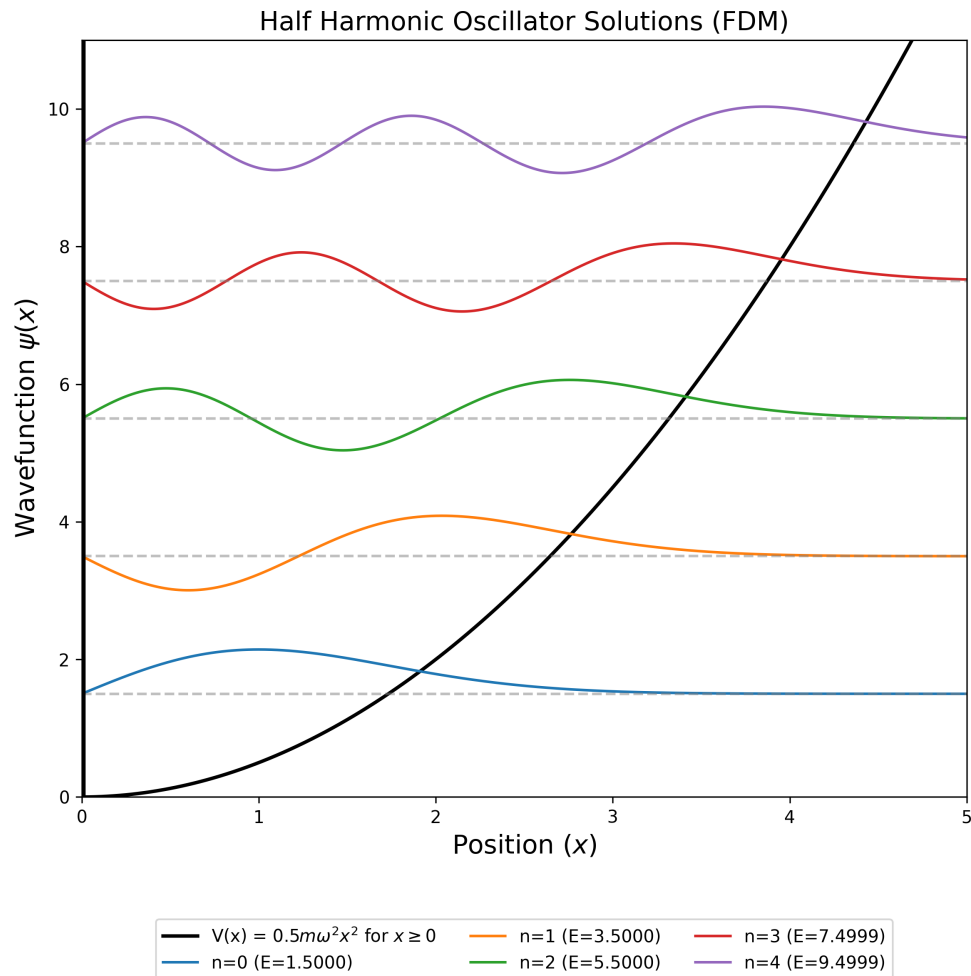
```

-----
Problem 2: Half Harmonic Oscillator
-----
Unit used: m = 1.0,  h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n = 1      E = 1.499996102
n = 2      E = 3.499980508
n = 3      E = 5.499952439
n = 4      E = 7.499911894
n = 5      E = 9.499858874
Wavefunctions written to prob02.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 10.0 to prevent overlap between the energy levels.



- **Plot Analysis:** The infinite wall forces  $\psi(0) = 0$ . Consequently, the valid solutions are exactly the odd-parity wavefunctions of the full harmonic oscillator. The full oscillator's even-parity ground state is destroyed.

### 3.3 Problem 3: One-dimensional Harmonic Potential

- **Description:** A standard harmonic oscillator.

- **Potential:**  $V(x) = \frac{1}{2}m\omega^2x^2$ .

- **Code:**

```

1  !=====
2  ! File      : prob03_harmonic_oscillator.f90
3  ! Author   : Mohak Das
4  ! Course   : Computational Lab: Quantum Mechanics
5  ! Topic    : FDM approximation for Harmonic Oscillator
6  ! Units    : m = h_bar = 1
7  ! Method   : Finite Difference Method
8  !=====
9
10
11 module prob03_physics

```

```

12  use constants_mod
13  implicit none
14  real(dp), parameter :: omega = 1.0_dp
15  integer, parameter :: N = 2001
16  real(dp) :: L = 5.0_dp
17  real(dp) :: x, dx
18  real(dp) :: d(N), e(N), z(N, N)
19  integer :: i
20
21  contains
22  subroutine initialize()
23      dx = (2.0_dp * L) / real(N - 1, dp)
24      z = 0.0_dp
25
26      do i = 1, N
27          z(i, i) = 1.0_dp
28      end do
29
30      do i = 1, N
31          x = - L + (i - 1) * dx
32          d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x)
33          if ( i < N ) then
34              e(i) = - (h_bar**2) / (2 * mass * dx**2)
35          else
36              e(N) = 0.0_dp
37          end if
38      end do
39  end subroutine initialize
40
41  function potential(x_vals) result(y)
42      real(dp), intent(in) :: x_vals
43      real(dp) :: y
44
45      y = 0.5_dp * mass * (omega**2) * (x_vals**2)
46  end function potential
47  end module prob03_physics
48
49  program prob03_driver
50      use FDM_toolbox
51      use prob03_physics
52      implicit none
53
54      call initialize()
55      call tqli(N, d, e, z)
56      call sort_eigen(N, d, z)
57
58      write(*, '(A)') "-----"
59      write(*, '(A)') "Problem 3: Harmonic Oscillator"
60      write(*, '(A)') "-----"
61      write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar
62      = ", h_bar
63      write(*, '(A)') "Method: Finite Difference Method"
64      write(*, '(A)') "-----"
65      "
66
67      write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'
68      ) " "

```

```

65
66     open(unit=10, file='data/prob03_energies.dat', status='replace')
67     do i = 1, 5
68         write(*, '(A, I2, A, G0.10)') "n = ", i-1, "      E = ", d(i)
69         write(10, *) d(i)
70     end do
71     close(10)
72
73     open(unit=10, file='data/prob03.dat', status='replace')
74     do i = 1, N
75         x = - L + (i - 1) * dx
76         write(10, *) x, potential(x), z(i,1), z(i,2), z(i,3), z(i, 4)
77         , z(i, 5)
78     end do
79     close(10)
80     print *, "Wavefunctions written to prob03.dat"
81 end program prob03_driver

```

- **Output:**

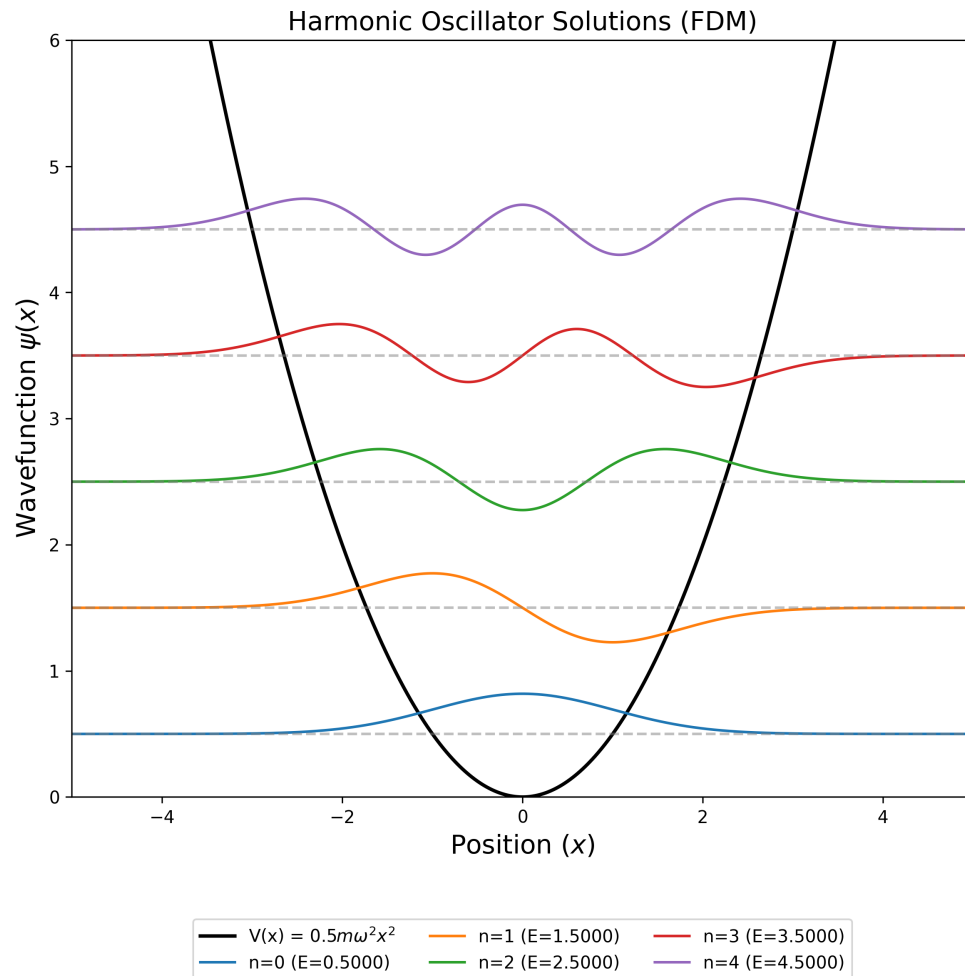
```

-----
Problem 3: Harmonic Oscillator
-----
Unit used: m = 1.0,  h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n =  0      E = 0.4999992188
n =  1      E = 1.499996097
n =  2      E = 2.499989926
n =  3      E = 3.499981665
n =  4      E = 4.499980359
Wavefunctions written to prob03.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 6.0 to prevent overlap between the energy levels.



- **Plot Analysis:** Yields the classic alternating parity states. The ground state is a perfect Gaussian bell curve centered at  $x = 0$ .

### 3.4 Problem 4: Finite Potential Well

- **Description:** A well with a finite depth  $V_0$ , allowing the particle to escape if  $E > V_0$ .
- **Potential:**  $V(x) = 0$  when  $-L \leq x \leq +L$ , and  $V_0$  elsewhere.
- **Code:**

```

1  !=====
2  ! File      : prob04_finite_square_well.f90
3  ! Author    : Mohak Das
4  ! Course    : Computational Lab: Quantum Mechanics
5  ! Topic     : FDM approximation for Harmonic Oscillator
6  ! Units     : m = h_bar = 1
7  ! Method    : Finite Difference Method
8  !=====
9
10
11 module prob04_physics
12   use constants_mod

```

```

13  implicit none
14  real(dp), parameter :: V0 = 200.0_dp
15  integer, parameter :: N = 2001
16  real(dp) :: L_well = 1.0_dp
17  real(dp) :: L_box = 4.0_dp
18  real(dp) :: x, dx
19  real(dp) :: d(N), e(N), z(N, N)
20  integer :: i
21
22  contains
23  subroutine initialize()
24      dx = (2.0_dp * L_box) / real(N - 1, dp)
25      z = 0.0_dp
26
27      do i = 1, N
28          z(i, i) = 1.0_dp
29      end do
30
31      do i = 1, N
32          x = - L_box + (i - 1) * dx
33          d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x)
34          if ( i < N ) then
35              e(i) = - (h_bar**2) / (2 * mass * dx**2)
36          else
37              e(N) = 0.0_dp
38          end if
39      end do
40  end subroutine initialize
41
42  function potential(x_vals) result(y)
43      real(dp), intent(in) :: x_vals
44      real(dp) :: y
45
46      if ( abs(x_vals) <= L_well ) then
47          y = 0.0_dp
48      else
49          y = V0
50      end if
51  end function potential
52 end module prob04_physics
53
54 program prob04_driver
55     use FDM_toolbox
56     use prob04_physics
57     implicit none
58
59     call initialize()
60     call tqli(N, d, e, z)
61     call sort_eigen(N, d, z)
62
63     write(*, '(A)') "-----"
64     write(*, '(A)') "Problem 4: Finite Square Well Potential"
65     write(*, '(A)') "-----"
66     write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar
67     = ", h_bar
68     write(*, '(A)') "Method: Finite Difference Method"

```

```

68     write(*, '(A)') "-----"
69     write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'
70     ) " "
71     open(unit=10, file='data/prob04_energies.dat', status='replace')
72     do i = 1, 5
73         write(*, '(A, I2, A, G0.10)') "n = ", i-1, "      E = ", d(i)
74         write(10, *) d(i)
75     end do
76     close(10)
77
78     open(unit=10, file='data/prob04.dat', status='replace')
79     do i = 1, N
80         x = - L_box + (i-1) * dx
81         write(10, *) x, potential(x), z(i,1), z(i,2), z(i,3), z(i, 4)
82         , z(i, 5)
83     end do
84     close(10)
85     print *, "Wavefunctions written to prob04.dat"
86 end program prob04_driver

```

### • Output:

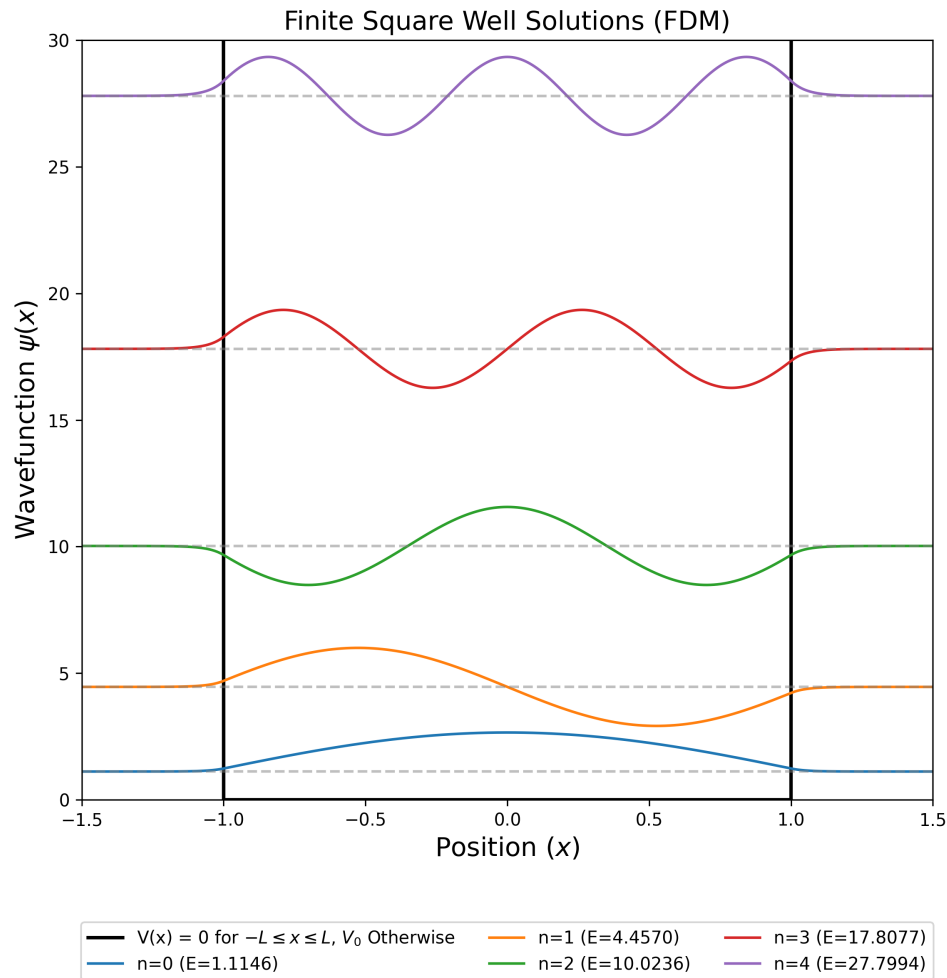
```

-----
Problem 4: Finite Square Well Potential
-----
Unit used: m = 1.0,  h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n =  0      E = 1.114565745
n =  1      E = 4.457032909
n =  2      E = 10.02362457
n =  3      E = 17.80774059
n =  4      E = 27.79943716
Wavefunctions written to prob04.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 25.0 to prevent overlap between the energy levels.



- **Plot Analysis:** The wavefunctions exhibit sinusoidal behavior inside the well but transition to exponential decay in the classically forbidden regions ( $|x| > L$ ). This demonstrates quantum penetration.

### 3.5 Problem 5: Parabolic Base Infinite Well

- **Description:** An infinite well where the floor curves upwards as a parabola.
- **Potential:**  $V(x) = \frac{V_0}{L^2}x^2$  for  $0 \leq x \leq L$ , and  $\infty$  elsewhere.
- **Code:**

```

1  !=====
2  ! File      : prob05_parabolic_base.f90
3  ! Author    : Mohak Das
4  ! Course    : Computational Lab: Quantum Mechanics
5  ! Topic     : FDM approximation for Infinite Potential Well With
   Parabolic Base
6  ! Units     : m = h_bar = 1
7  ! Method    : Finite Difference Method
8  !=====
9
10

```

```

11 module prob05_physics
12     use constants_mod
13     implicit none
14     real(dp), parameter :: V0 = 20000.0_dp
15     integer, parameter :: N = 2001
16     real(dp) :: L_well = 1.0_dp
17     real(dp) :: L_box = 4.0_dp
18     real(dp) :: x, dx
19     real(dp) :: d(N), e(N), z(N, N)
20     integer :: i
21
22 contains
23     subroutine initialize()
24         dx = L_box / real(N + 1, dp)
25         z = 0.0_dp
26
27         do i = 1, N
28             z(i, i) = 1.0_dp
29         end do
30
31         do i = 1, N
32             x = real(i, dp) * dx
33             d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x)
34
35             if ( i < N ) then
36                 e(i) = - (h_bar**2) / (2 * mass * dx**2)
37             else
38                 e(N) = 0.0_dp
39             end if
40         end do
41     end subroutine initialize
42
43     function potential(x_vals) result(y)
44         real(dp), intent(in) :: x_vals
45         real(dp) :: y
46
47         y = (V0/(L_well**2)) * (x_vals**2)
48     end function potential
49 end module prob05_physics
50
51 program prob05_driver
52     use FDM_toolbox
53     use prob05_physics
54     implicit none
55
56     call initialize()
57     call tqli(N, d, e, z)
58     call sort_eigen(N, d, z)
59
60     write(*, '(A)') "
61     -----"
62     write(*, '(A)') "Problem 5: Infinite Potential Well With
63     Parabolic Base"
64     write(*, '(A)') "
65     -----"
66     write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar

```

```

        = ", h_bar
64 write(*, '(A)') "Method: Finite Difference Method"
65 write(*, '(A)') "
-----"
66 write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'
) " "
67
68 open(unit=10, file='data/prob05_energies.dat', status='replace')
69 do i = 1, 5
70     write(*, '(A, I2, A, G0.10)') "n = ", i, "      E = ", d(i)
71     write(10, *) d(i)
72 end do
73 close(10)
74
75 open(unit=10, file='data/prob05.dat', status='replace')
76 do i = 1, N
77     x = real(i, dp) * dx
78     write(10, *) x, potential(x), z(i,1), z(i, 2), z(i, 3), z(i,
4), z(i, 5)
79 end do
80 close(10)
81 write(*, '(A)') "Wavefunctions written to prob05.dat"
82 end program prob05_driver

```

### • Output:

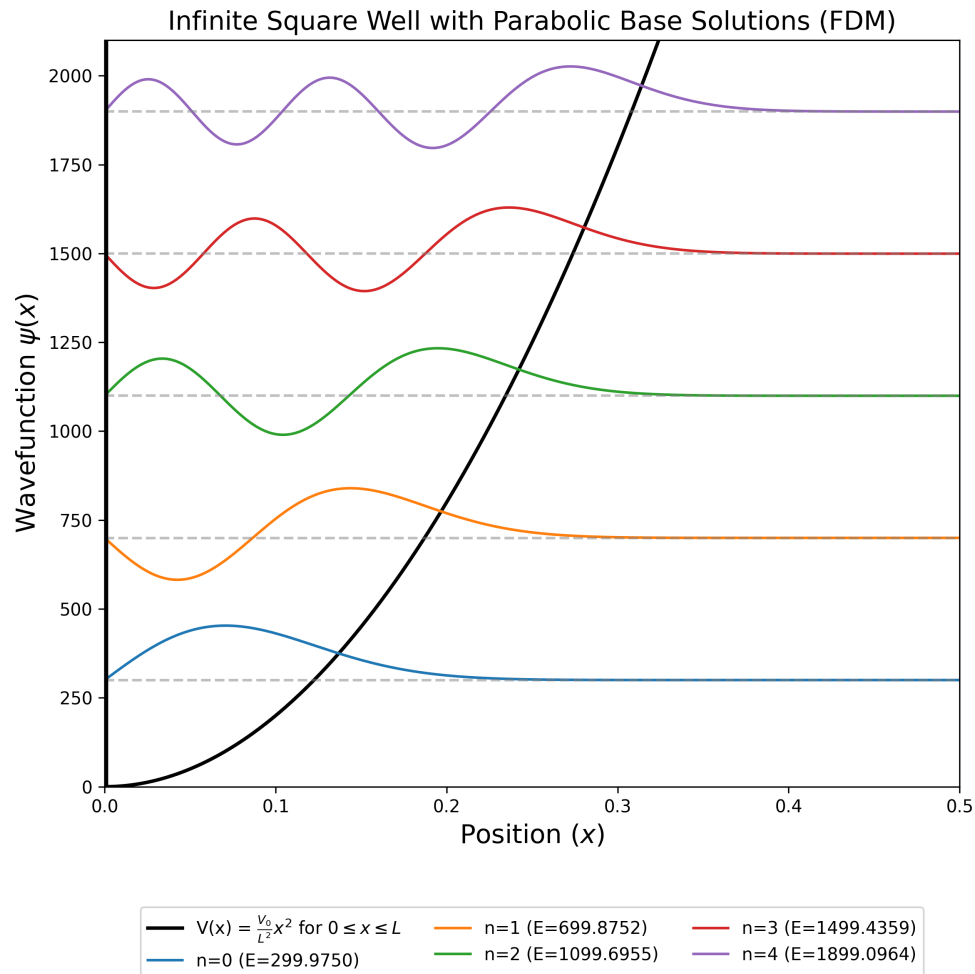
```

-----
Problem 5: Infinite Potential Well With Parabolic Base
-----
Unit used: m = 1.0,  h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n =  1      E = 299.9750477
n =  2      E = 699.8752270
n =  3      E = 1099.695524
n =  4      E = 1499.435915
n =  5      E = 1899.096376
Wavefunctions written to prob05.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 1000.0 to prevent overlap between the energy levels.



- **Plot Analysis:** Combines features of the harmonic oscillator and the infinite well. The wavefunctions are squeezed inward by the parabolic floor but forcefully truncated at the infinite boundaries.

### 3.6 Problem 6: Modulo $x$ Potential

- **Description:** A V-shaped linear potential well.
- **Potential:**  $V(x) = |x|$ .
- **Code:**

```

1  !=====
2  ! File      : prob06_mod_x.f90
3  ! Author   : Mohak Das
4  ! Course   : Computational Lab: Quantum Mechanics
5  ! Topic    : FDM approximation for |x| potential
6  ! Units    : m = h_bar = 1
7  ! Method   : Finite Difference Method
8  !=====
9
10
11 module prob06_physics

```

```

12  use constants_mod
13  implicit none
14  integer, parameter :: N = 2001
15  real(dp) :: L_box = 7.0_dp
16  real(dp) :: x, dx
17  real(dp) :: d(N), e(N), z(N, N)
18  integer :: i
19
20  contains
21  subroutine initialize()
22      dx = (2 * L_box) / real(N - 1, dp)
23      z = 0.0_dp
24
25      do i = 1, N
26          z(i, i) = 1.0_dp
27      end do
28
29      do i = 1, N
30          x = - L_box + real(i - 1, dp) * dx
31          d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x)
32          if ( i < N ) then
33              e(i) = - (h_bar**2) / (2 * mass * dx**2)
34          else
35              e(N) = 0.0_dp
36          end if
37      end do
38  end subroutine initialize
39
40  function potential(x_vals) result(y)
41      real(dp), intent(in) :: x_vals
42      real(dp) :: y
43
44      y = abs(x_vals)
45  end function potential
46  end module prob06_physics
47
48  program prob06_driver
49      use FDM_toolbox
50      use prob06_physics
51      implicit none
52
53      call initialize()
54      call tqli(N, d, e, z)
55      call sort_eigen(N, d, z)
56
57      write(*, '(A)') "-----"
58      write(*, '(A)') "Problem 6: |x| potential"
59      write(*, '(A)') "-----"
60      write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar
61      = ", h_bar
62      write(*, '(A)') "Method: Finite Difference Method"
63      write(*, '(A)') "-----"
64      write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'
65      ) " "
66
67      open(unit=10, file='data/prob06_energies.dat', status='replace')

```

```

66     do i = 1, 5
67         write(*, '(A, I2, A, G0.10)') "n = ", i-1, "      E = ", d(i)
68         write(10, *) d(i)
69     end do
70     close(10)
71
72     open(unit=10, file='data/prob06.dat', status='replace')
73     do i = 1, N
74         x = - L_box + real(i-1, dp) * dx
75         write(10, *) x, potential(x), z(i,1), z(i,2), z(i,3), z(i, 4)
76         , z(i, 5)
77     end do
78     close(10)
79     print *, "Wavefunctions written to prob06.dat"
80 end program prob06_driver

```

### • Output:

```

-----
Problem 6: |x| potential
-----

```

```

Unit used: m = 1.0,  h_bar = 1.0
Method: Finite Difference Method
-----

```

```

The first five eigen energies:

```

```

n = 0      E = 0.8086093897
n = 1      E = 1.855751457
n = 2      E = 2.578083378
n = 3      E = 3.244590636
n = 4      E = 3.825694142

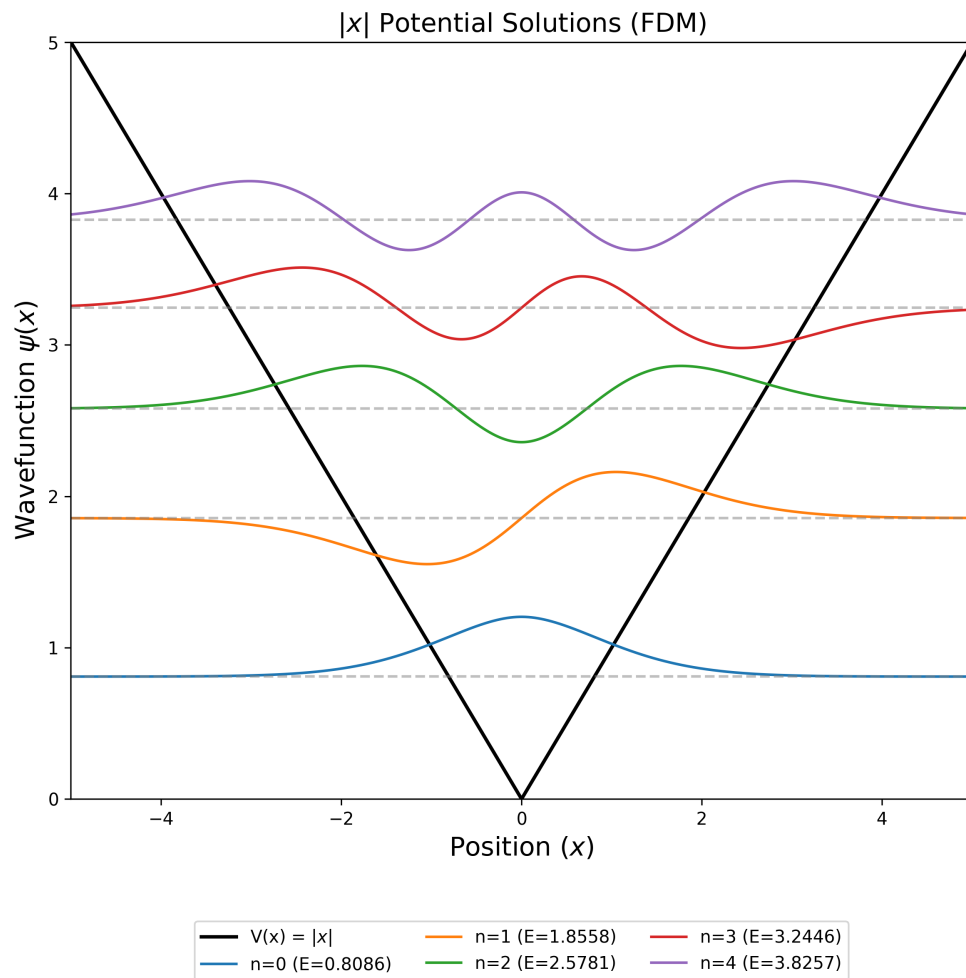
```

```

Wavefunctions written to prob06.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 6.0 to prevent overlap between the energy levels.



- **Plot Analysis:** The states are tightly bound at the bottom of the "V". Because the potential rises linearly rather than quadratically, the energy levels are spaced more closely together than in the harmonic oscillator.

### 3.7 Problem 7: Modulo $x^3$ Potential

- **Description:** A steep, non-harmonic symmetric well.
- **Potential:**  $V(x) = |x^3|$ .
- **Code:**

```

1  !=====
2  ! File      : prob07_mod_x_cube.f90
3  ! Author    : Mohak Das
4  ! Course    : Computational Lab: Quantum Mechanics
5  ! Topic     : FDM approximation for |x^3| potential
6  ! Units     : m = h_bar = 1
7  ! Method    : Finite Difference Method
8  !=====
9
10
11 module prob07_physics

```

```

12     use constants_mod
13     implicit none
14     integer, parameter :: N = 2001
15     real(dp) :: L_box = 4.0_dp
16     real(dp) :: x, dx
17     real(dp) :: d(N), e(N), z(N, N)
18     integer :: i
19
20 contains
21     subroutine initialize()
22         dx = (2 * L_box) / real(N - 1, dp)
23         z = 0.0_dp
24
25         do i = 1, N
26             z(i, i) = 1.0_dp
27         end do
28
29         do i = 1, N
30             x = - L_box + real(i - 1, dp) * dx
31             d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x)
32             if ( i < N ) then
33                 e(i) = - (h_bar**2) / (2 * mass * dx**2)
34             else
35                 e(N) = 0.0_dp
36             end if
37         end do
38     end subroutine initialize
39
40     function potential(x_vals) result(y)
41         real(dp), intent(in) :: x_vals
42         real(dp) :: y
43
44         y = abs(x_vals**3)
45     end function potential
46 end module prob07_physics
47
48 program prob07_driver
49     use FDM_toolbox
50     use prob07_physics
51     implicit none
52
53     call initialize()
54     call tqli(N, d, e, z)
55     call sort_eigen(N, d, z)
56
57     write(*, '(A)') "-----"
58     write(*, '(A)') "Problem 7: |x^3| potential"
59     write(*, '(A)') "-----"
60     write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar
61     = ", h_bar
62     write(*, '(A)') "Method: Finite Difference Method"
63     write(*, '(A)') "-----"
64     write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'
65 ) " "
66
67     open(unit=10, file='data/prob07_energies.dat', status='replace')

```

```

66     do i = 1, 5
67         write(*, '(A, I2, A, G0.10)') "n = ", i-1, "      E = ", d(i)
68         write(10, *) d(i)
69     end do
70     close(10)
71
72     open(unit=10, file='data/prob07.dat', status='replace')
73     do i = 1, N
74         x = - L_box + real(i-1, dp) * dx
75         write(10, *) x, potential(x), z(i,1), z(i,2), z(i,3), z(i, 4)
76         , z(i, 5)
77     end do
78     close(10)
79     print *, "Wavefunctions written to prob07.dat"
80 end program prob07_driver

```

- **Output:**

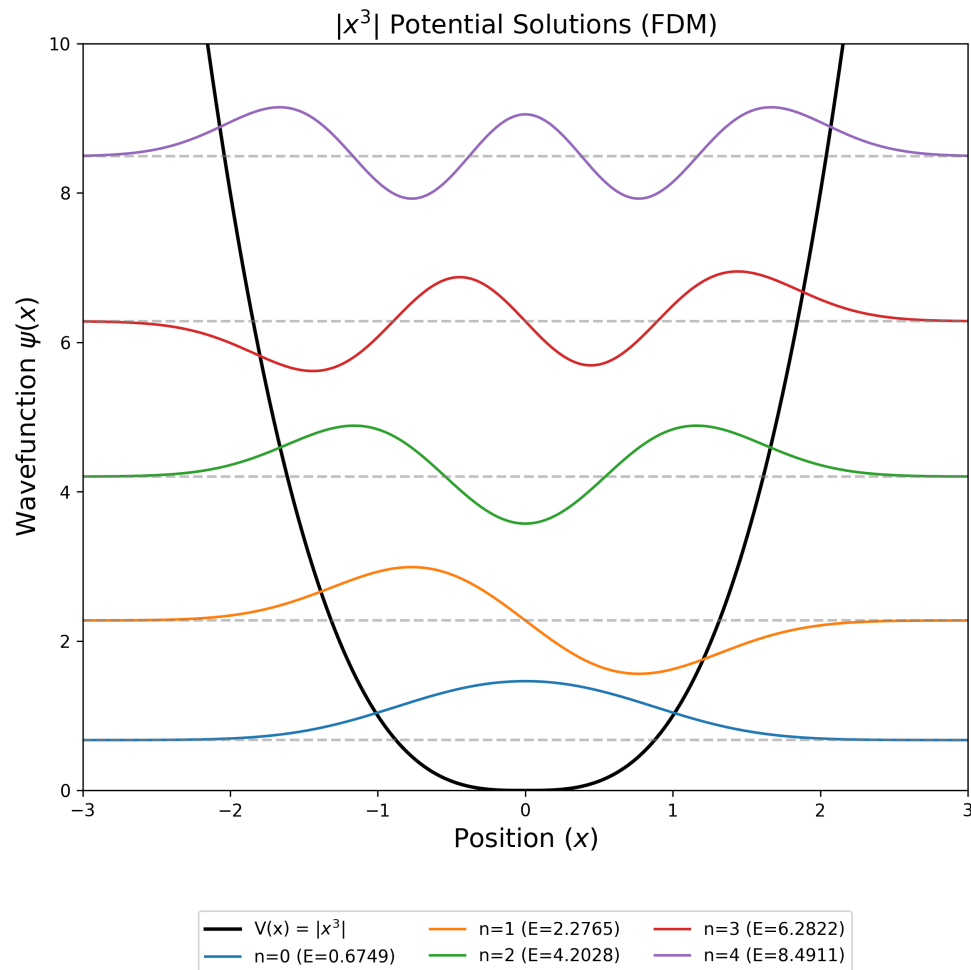
```

-----
Problem 7: |x^3| potential
-----
Unit used: m = 1.0,  h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n =  0      E = 0.6748926574
n =  1      E = 2.276514372
n =  2      E = 4.202801600
n =  3      E = 6.282174284
n =  4      E = 8.491132914
Wavefunctions written to prob07.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 15.0 to prevent overlap between the energy levels.



- **Plot Analysis:** The potential walls rise much faster than a parabola. This tighter spatial confinement leads to energy levels that spread further apart as  $n$  increases, contrasting with the evenly spaced levels of the harmonic oscillator.

### 3.8 Problem 8: Quartic Potential

- **Description:** A symmetric well with walls that rise as  $x^4$ .
- **Potential:**  $V(x) = x^4$ .
- **Code:**

```

1  !=====
2  ! File      : prob08_quartic_potential.f90
3  ! Author    : Mohak Das
4  ! Course    : Computational Lab: Quantum Mechanics
5  ! Topic     : FDM approximation for Quartic (x^4) potential
6  ! Units     : m = h_bar = 1
7  ! Method    : Finite Difference Method
8  !=====
9
10
11 module prob08_physics

```

```

12  use constants_mod
13  implicit none
14  integer, parameter :: N = 2001
15  real(dp) :: L_box = 4.0_dp
16  real(dp) :: x, dx
17  real(dp) :: d(N), e(N), z(N, N)
18  integer :: i
19
20  contains
21  subroutine initialize()
22      dx = (2 * L_box) / real(N - 1, dp)
23      z = 0.0_dp
24
25      do i = 1, N
26          z(i, i) = 1.0_dp
27      end do
28
29      do i = 1, N
30          x = - L_box + real(i - 1, dp) * dx
31          d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x)
32          if ( i < N ) then
33              e(i) = - (h_bar**2) / (2 * mass * dx**2)
34          else
35              e(N) = 0.0_dp
36          end if
37      end do
38  end subroutine initialize
39
40  function potential(x_vals) result(y)
41      real(dp), intent(in) :: x_vals
42      real(dp) :: y
43
44      y = x_vals**4
45  end function potential
46  end module prob08_physics
47
48  program prob08_driver
49      use FDM_toolbox
50      use prob08_physics
51      implicit none
52
53      call initialize()
54      call tqli(N, d, e, z)
55      call sort_eigen(N, d, z)
56
57      write(*, '(A)') "-----"
58      "
59      write(*, '(A)') "Problem 8: Quartic (x^4) Potential"
60      write(*, '(A)') "-----"
61      "
62      write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar
63      = ", h_bar
64      write(*, '(A)') "Method: Finite Difference Method"
65      write(*, '(A)') "-----"
66      "
67      write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'

```

```

) " "
64
65     open(unit=10, file='data/prob08_energies.dat', status='replace')
66     do i = 1, 5
67         write(*, '(A, I2, A, G0.10)') "n = ", i-1, "      E = ", d(i)
68         write(10, *) d(i)
69     end do
70     close(10)
71
72     open(unit=10, file='data/prob08.dat', status='replace')
73     do i = 1, N
74         x = - L_box + real(i-1, dp) * dx
75         write(10, *) x, potential(x), z(i,1), z(i,2), z(i,3), z(i, 4)
76     , z(i, 5)
77     end do
78     close(10)
79     print *, "Wavefunctions written to prob08.dat"
end program prob08_driver

```

### • Output:

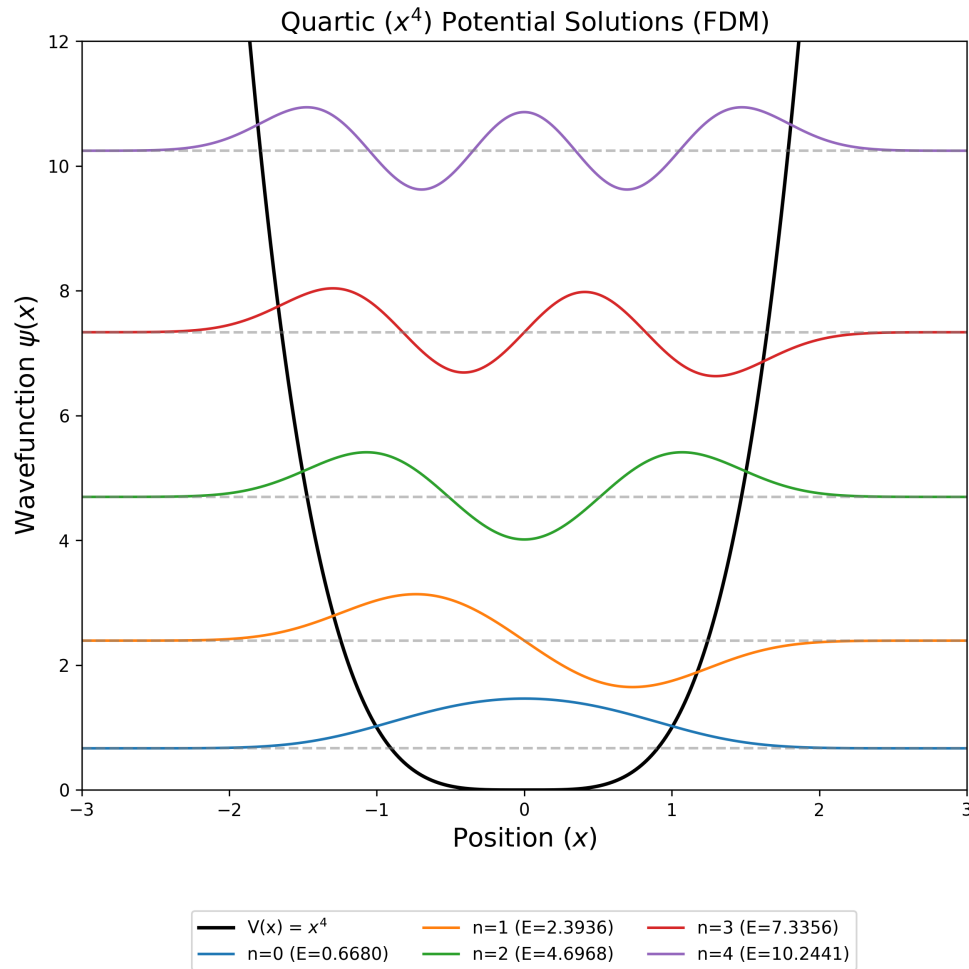
```

-----
Problem 8: Quartic (x^4) Potential
-----
Unit used: m = 1.0,  h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n =  0      E = 0.6679847582
n =  1      E = 2.393633241
n =  2      E = 4.696758949
n =  3      E = 7.335644461
n =  4      E = 10.24414436
Wavefunctions written to prob08.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 15.0 to prevent overlap between the energy levels.



- **Plot Analysis:** Extremely flat at the bottom but incredibly steep at the edges. The wavefunctions near the ground state spread out over the flat bottom, while higher states are abruptly reflected by the steep quartic walls.

### 3.9 Problem 9: Quadratic-Quartic Potential

- **Description:** An anharmonic oscillator combining both  $x^2$  and  $x^4$  terms.
- **Potential:**  $V(x) = x^2 + x^4$ .
- **Code:**

```

1  !=====
2  ! File      : prob09_quartic_quadratic.f90
3  ! Author    : Mohak Das
4  ! Course    : Computational Lab: Quantum Mechanics
5  ! Topic     : FDM approximation for Quadratic-quartic potential
6  ! Units     : m = h_bar = 1
7  ! Method    : Finite Difference Method
8  !=====
9
10
11 module prob09_physics

```

```

12     use constants_mod
13     implicit none
14     integer, parameter :: N = 2001
15     real(dp) :: L_box = 4.0_dp
16     real(dp) :: x, dx
17     real(dp) :: d(N), e(N), z(N, N)
18     integer :: i
19
20 contains
21     subroutine initialize()
22         dx = (2 * L_box) / real(N - 1, dp)
23         z = 0.0_dp
24
25         do i = 1, N
26             z(i, i) = 1.0_dp
27         end do
28
29         do i = 1, N
30             x = - L_box + real(i - 1, dp) * dx
31             d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x)
32             if ( i < N ) then
33                 e(i) = - (h_bar**2) / (2 * mass * dx**2)
34             else
35                 e(N) = 0.0_dp
36             end if
37         end do
38     end subroutine initialize
39
40     function potential(x_vals) result(y)
41         real(dp), intent(in) :: x_vals
42         real(dp) :: y
43
44         y = x_vals**2 + x_vals**4
45     end function potential
46 end module prob09_physics
47
48 program prob09_driver
49     use FDM_toolbox
50     use prob09_physics
51     implicit none
52
53     call initialize()
54     call tqli(N, d, e, z)
55     call sort_eigen(N, d, z)
56
57     write(*, '(A)') "-----"
58     "
59     write(*, '(A)') "Problem 9: Quadratic-quartic potential"
60     write(*, '(A)') "-----"
61     "
62     write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar
63     = ", h_bar
64     write(*, '(A)') "Method: Finite Difference Method"
65     write(*, '(A)') "-----"
66     "
67     write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'

```

```

) " "
64
65 open(unit=10, file='data/prob09_energies.dat', status='replace')
66 do i = 1, 5
67     write(*, '(A, I2, A, G0.10)') "n = ", i-1, "      E = ", d(i)
68     write(10, *) d(i)
69 end do
70 close(10)
71
72 open(unit=10, file='data/prob09.dat', status='replace')
73 do i = 1, N
74     x = - L_box + real(i-1, dp) * dx
75     write(10, *) x, potential(x), z(i,1), z(i,2), z(i,3), z(i, 4)
76 , z(i, 5)
76 end do
77 close(10)
78 print *, "Wavefunctions written to prob09.dat"
79 end program prob09_driver

```

### • Output:

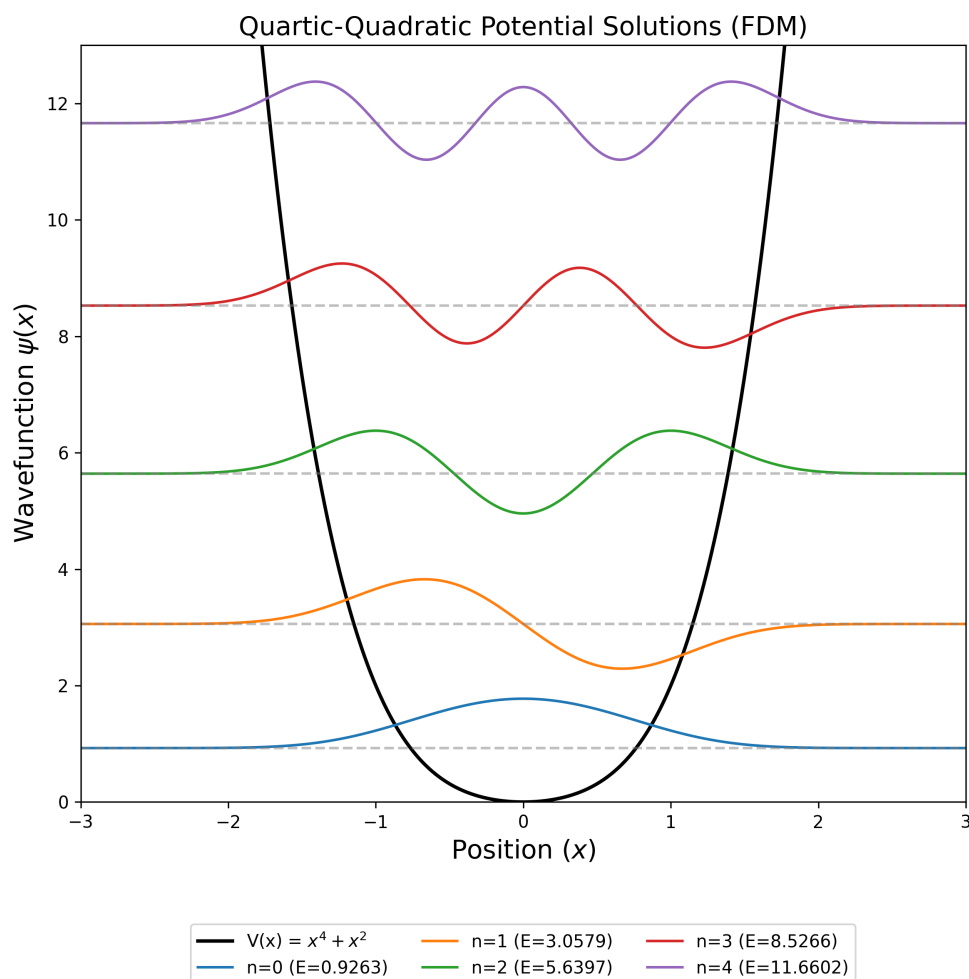
```

-----
Problem 9: Quadratic-quartic potential
-----
Unit used: m = 1.0,  h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n =  0      E = 0.9262562307
n =  1      E = 3.057931421
n =  2      E = 5.639704502
n =  3      E = 8.526634317
n =  4      E = 11.66020884
Wavefunctions written to prob09.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 15.0 to prevent overlap between the energy levels.



- **Plot Analysis:** The  $x^2$  term dominates near the origin, making the ground state resemble a standard harmonic oscillator. For higher energy states, the  $x^4$  term dominates, causing the energy gaps to increase rapidly.

### 3.10 Problem 10: Half-Cubic Potential

- **Description:** A cubic well bounded by an infinite wall at the origin.
- **Potential:**  $V(x) = x^3$  when  $x \geq 0$ , and  $\infty$  otherwise.
- **Code:**

```

1  !=====
2  ! File      : prob10_half_cubic.f90
3  ! Author    : Mohak Das
4  ! Course    : Computational Lab: Quantum Mechanics
5  ! Topic     : FDM approximation for Half-cubic potential
6  ! Units     : m = h_bar = 1
7  ! Method    : Finite Difference Method
8  !=====
9
10
11 module prob10_physics

```

```

12  use constants_mod
13  implicit none
14  integer, parameter :: N = 2001
15  real(dp) :: L_box = 4.0_dp
16  real(dp) :: x, dx
17  real(dp) :: d(N), e(N), z(N, N)
18  integer :: i
19
20  contains
21  subroutine initialize()
22      dx = L_box / real(N + 1, dp)
23      z = 0.0_dp
24
25      do i = 1, N
26          z(i, i) = 1.0_dp
27      end do
28
29      do i = 1, N
30          x = real(i, dp) * dx
31          d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x)
32          if ( i < N ) then
33              e(i) = - (h_bar**2) / (2 * mass * dx**2)
34          else
35              e(N) = 0.0_dp
36          end if
37      end do
38  end subroutine initialize
39
40  function potential(x_vals) result(y)
41      real(dp), intent(in) :: x_vals
42      real(dp) :: y
43
44      y = x_vals**3
45  end function potential
46  end module prob10_physics
47
48  program prob10_driver
49      use FDM_toolbox
50      use prob10_physics
51      implicit none
52
53      call initialize()
54      call tqli(N, d, e, z)
55      call sort_eigen(N, d, z)
56
57      write(*, '(A)') "-----"
58      "
59      write(*, '(A)') "Problem 10: Half-cubic Potential"
60      write(*, '(A)') "-----"
61      "
62      write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar
63      = ", h_bar
64      write(*, '(A)') "Method: Finite Difference Method"
65      write(*, '(A)') "-----"
66      "
67      write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'

```

```

) " "
64
65 open(unit=10, file='data/prob10_energies.dat', status='replace')
66 do i = 1, 5
67     write(*, '(A, I2, A, G0.10)') "n = ", i, "      E = ", d(i)
68     write(10, *) d(i)
69 end do
70 close(10)
71
72 open(unit=10, file='data/prob10.dat', status='replace')
73 do i = 1, N
74     x = real(i, dp) * dx
75     write(10, *) x, potential(x), z(i,1), z(i, 2), z(i, 3), z(i,
4), z(i, 5)
76 end do
77 close(10)
78 write(*, '(A)') "Wavefunctions written to prob10.dat"
79 end program prob10_driver

```

### • Output:

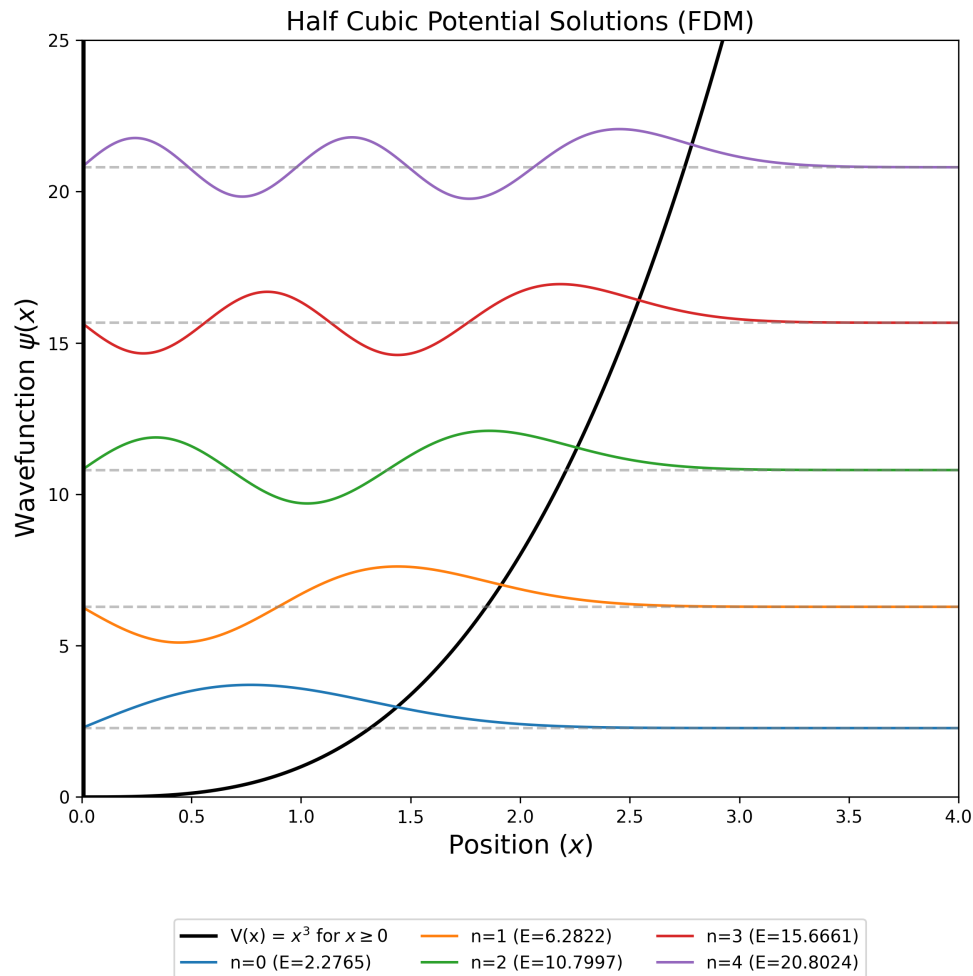
```

-----
Problem 10: Half-cubic Potential
-----
Unit used: m = 1.0,  h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n = 1      E = 2.276520384
n = 2      E = 6.282214306
n = 3      E = 10.79971970
n = 4      E = 15.66608800
n = 5      E = 20.80242178
Wavefunctions written to prob10.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 30.0 to prevent overlap between the energy levels.



- **Plot Analysis:** Similar to the half-harmonic case, the wavefunctions are strictly zero at  $x = 0$ . The asymmetry of the  $x^3$  growth pushes the peaks of the higher excited states further to the right compared to a quadratic well.

### 3.11 Problem 11: Half-Quartic Potential

- **Description:** A quartic well bounded by an infinite wall at the origin.
- **Potential:**  $V(x) = x^4$  when  $x \geq 0$ , and  $\infty$  otherwise.
- **Code:**

```

1  !=====
2  ! File      : prob11_half_quartic.f90
3  ! Author    : Mohak Das
4  ! Course    : Computational Lab: Quantum Mechanics
5  ! Topic     : FDM approximation for Half-quartic potential
6  ! Units     : m = h_bar = 1
7  ! Method    : Finite Difference Method
8  !=====
9
10
11 module prob11_physics

```

```

12     use constants_mod
13     implicit none
14     integer, parameter :: N = 2001
15     real(dp) :: L_box = 4.0_dp
16     real(dp) :: x, dx
17     real(dp) :: d(N), e(N), z(N, N)
18     integer :: i
19
20 contains
21     subroutine initialize()
22         dx = L_box / real(N + 1, dp)
23         z = 0.0_dp
24
25         do i = 1, N
26             z(i, i) = 1.0_dp
27         end do
28
29         do i = 1, N
30             x = real(i, dp) * dx
31             d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x)
32             if ( i < N ) then
33                 e(i) = - (h_bar**2) / (2 * mass * dx**2)
34             else
35                 e(N) = 0.0_dp
36             end if
37         end do
38     end subroutine initialize
39
40     function potential(x_vals) result(y)
41         real(dp), intent(in) :: x_vals
42         real(dp) :: y
43
44         y = x_vals**4
45     end function potential
46 end module prob11_physics
47
48 program prob11_driver
49     use FDM_toolbox
50     use prob11_physics
51     implicit none
52
53     call initialize()
54     call tqli(N, d, e, z)
55     call sort_eigen(N, d, z)
56
57     write(*, '(A)') "-----"
58     write(*, '(A)') "Problem 11: Half Quartic potential"
59     write(*, '(A)') "-----"
60     write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar
61     = ", h_bar
62     write(*, '(A)') "Method: Finite Difference Method"
63     write(*, '(A)') "-----"
64     write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'
65 ) " "
66
67     open(unit=10, file='data/prob11_energies.dat', status='replace')

```

```

66     do i = 1, 5
67         write(*, '(A, I2, A, G0.10)') "n = ", i, "      E = ", d(i)
68         write(10, *) d(i)
69     end do
70     close(10)
71
72     open(unit=10, file='data/prob11.dat', status='replace')
73     do i = 1, N
74         x = real(i, dp) * dx
75         write(10, *) x, potential(x), z(i,1), z(i, 2), z(i, 3), z(i,
76         4), z(i, 5)
77     end do
78     close(10)
79     write(*, '(A)') "Wavefunctions written to prob11.dat"
80 end program prob11_driver

```

- **Output:**

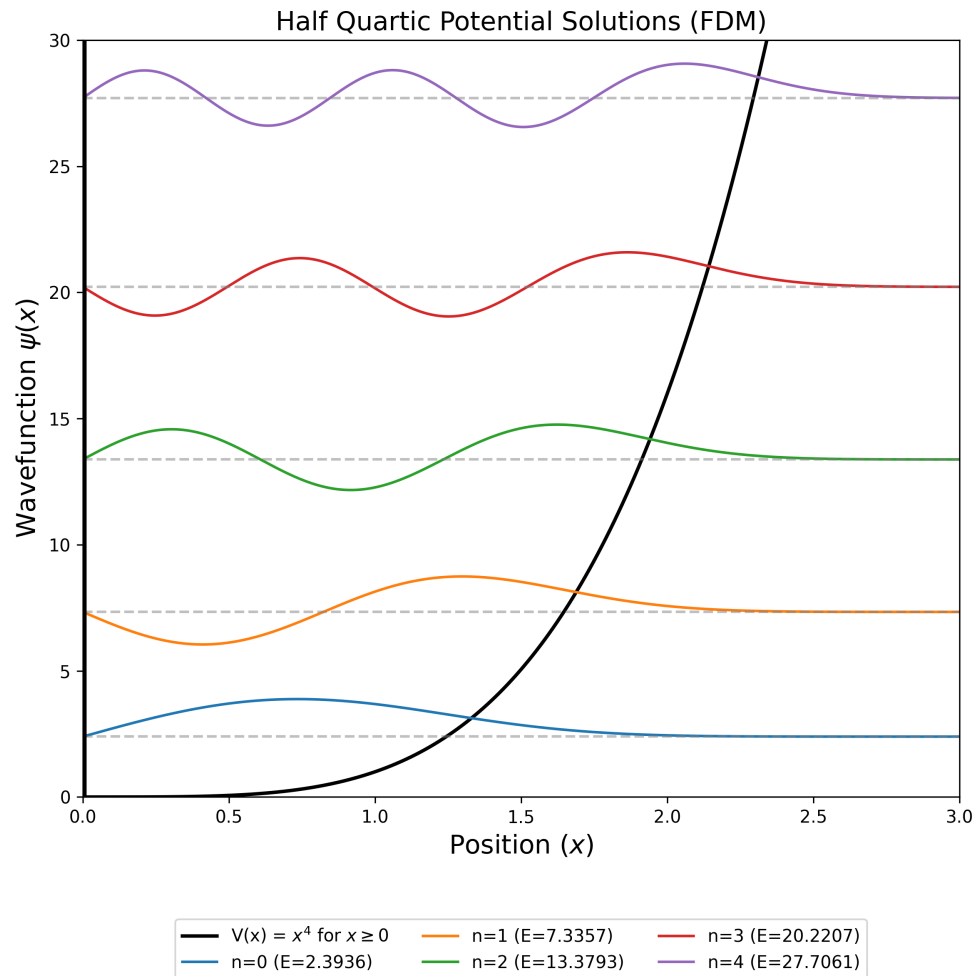
```

-----
Problem 11: Half Quartic potential
-----
Unit used: m = 1.0,  h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n =  1      E = 2.393641328
n =  2      E = 7.335708655
n =  3      E = 13.37926730
n =  4      E = 20.22069255
n =  5      E = 27.70609988
Wavefunctions written to prob11.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 30.0 to prevent overlap between the energy levels.



- **Plot Analysis:** Features a highly compressed spatial domain due to the steep  $x^4$  rise combined with the hard wall. This leads to highly localized probability densities and widely separated energy eigenvalues.

## 4 Additional Exercises

### 4.1 Problem 12: Double Well Potential

- **Description:** Two deep minima separated by a central potential barrier.
- **Potential:**  $V(x) = \lambda(x^2 - a^2)^2$  (Used:  $V(x) = 0.5(x^2 - 9)^2$ ).
- **Code:**

```

1  !=====
2  ! File      : prob12_double_well.f90 (Additional Exercise)
3  ! Author    : Mohak Das
4  ! Course    : Computational Lab: Quantum Mechanics
5  ! Topic     : FDM approximation for Double Well potential
6  ! Units     : m = h_bar = 1
7  ! Method    : Finite Difference Method
8  !=====

```

```

9
10
11 module prob12_physics
12     use constants_mod
13     implicit none
14     integer, parameter :: N = 2001
15     real(dp), parameter :: lambda = 0.5_dp
16     real(dp), parameter :: a = 3.0_dp
17     real(dp) :: L_box = 6.0_dp
18     real(dp) :: x, dx
19     real(dp) :: d(N), e(N), z(N, N)
20     integer :: i
21
22 contains
23     subroutine initialize()
24         dx = (2 * L_box) / real(N - 1, dp)
25         z = 0.0_dp
26
27         do i = 1, N
28             z(i, i) = 1.0_dp
29         end do
30
31         do i = 1, N
32             x = - L_box + real(i - 1, dp) * dx
33             d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x)
34             if ( i < N ) then
35                 e(i) = - (h_bar**2) / (2 * mass * dx**2)
36             else
37                 e(N) = 0.0_dp
38             end if
39         end do
40     end subroutine initialize
41
42     function potential(x_vals) result(y)
43         real(dp), intent(in) :: x_vals
44         real(dp) :: y
45
46         y = lambda * (x_vals**2 - a**2)**2
47     end function potential
48 end module prob12_physics
49
50 program prob12_driver
51     use FDM_toolbox
52     use prob12_physics
53     implicit none
54
55     call initialize()
56     call tqli(N, d, e, z)
57     call sort_eigen(N, d, z)
58
59     write(*, '(A)') "-----"
60     write(*, '(A)') "Problem 12: Double Well potential"
61     write(*, '(A)') "-----"
62     write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar

```

```

= ", h_bar
63 write(*, '(A)') "Method: Finite Difference Method"
64 write(*, '(A)') "-----"
"
65 write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'
) " "
66
67 open(unit=10, file='data/prob12_energies.dat', status='replace')
68 do i = 1, 5
69     write(*, '(A, I2, A, G0.10)') "n = ", i-1, "      E = ", d(i)
70     write(10, *) d(i)
71 end do
72 close(10)
73
74 write(*, '(A, G0.10)') "Tunneling Splitting (Delta E): ", d(2) -
d(1)
75
76 open(unit=10, file='data/prob12.dat', status='replace')
77 do i = 1, N
78     x = - L_box + real(i-1, dp) * dx
79     write(10, *) x, potential(x), z(i,1), z(i,2), z(i,3), z(i, 4)
, z(i, 5)
80 end do
81 close(10)
82 print *, "Wavefunctions written to prob12.dat"
83 end program prob12_driver

```

#### • Output:

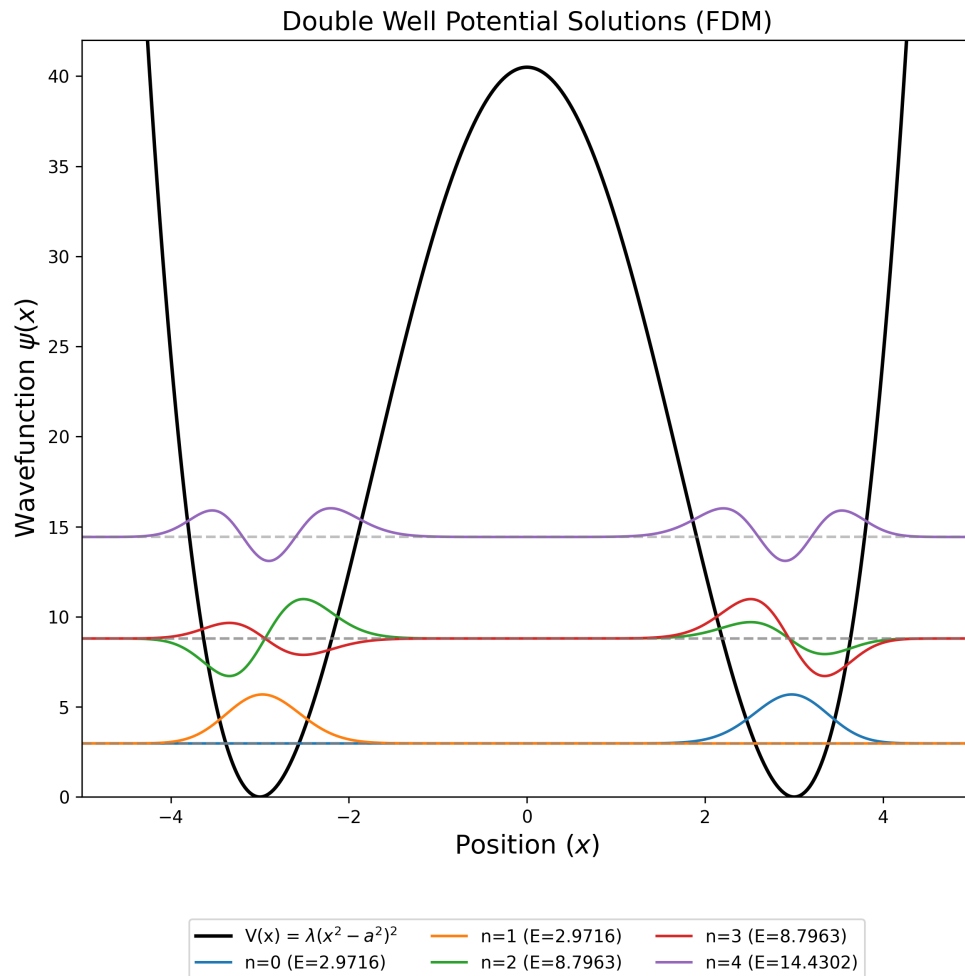
```

-----
Problem 12: Double Well potential
-----
Unit used: m = 1.0,  h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n =  0      E = 2.971575787
n =  1      E = 2.971575787
n =  2      E = 8.796253494
n =  3      E = 8.796253494
n =  4      E = 14.43020712
Tunneling Splitting (Delta E): 0.1456434973E-10
Wavefunctions written to prob12.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 30.0 to prevent overlap between the energy levels.



- **Plot Analysis:** Reveals quantum tunneling. The lowest energy states form almost-degenerate pairs ( $E_0 \approx E_1$ ). The splitting  $\Delta E$  is infinitesimally small. Due to the limits of double-precision floating-point arithmetic, the solver exhibits spontaneous symmetry breaking, resolving the states into localized wavefunctions in either the left or right well rather than perfect symmetric/antisymmetric superpositions.

## 4.2 Problem 13: The Morse Potential

- **Description:** The Morse potential is a realistic model for the vibrational structure of a diatomic molecule. Unlike the simple harmonic oscillator, which assumes bonds can stretch infinitely without breaking, the Morse potential accurately models molecular dissociation. It features a steep repulsive wall at short distances and flattens out to a finite dissociation energy at large distances.
- **Potential:**  $V(x) = D_e(1 - e^{-ax})^2$  (Used:  $V(x) = 2(1 - e^{-0.18x})^2$ ).
- **Code:**

```

1 !=====
2 ! File      : prob13_morse_potential.f90 (Additional Exercise)
3 ! Author   : Mohak Das
4 ! Course   : Computational Lab: Quantum Mechanics

```

```

5 ! Topic    : FDM approximation for Morse potential
6 ! Units    :  $m = \hbar = 1$ 
7 ! Method   : Finite Difference Method
8 ! =====
9
10
11 module prob13_physics
12     use constants_mod
13     implicit none
14     integer, parameter :: N = 2001
15     real(dp), parameter :: lambda = 2
16     real(dp), parameter :: a = 0.18
17     real(dp) :: L_box_r = 40.0_dp
18     real(dp) :: L_box_l = -6.0_dp
19     real(dp) :: x, dx
20     real(dp) :: d(N), e(N), z(N, N)
21     integer :: i
22
23 contains
24     subroutine initialize()
25         dx = (L_box_r - L_box_l) / real(N - 1, dp)
26         z = 0.0_dp
27
28         do i = 1, N
29             z(i, i) = 1.0_dp
30         end do
31
32         do i = 1, N
33             x = L_box_l + real(i - 1, dp) * dx
34             d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x)
35             if ( i < N ) then
36                 e(i) = - (h_bar**2) / (2 * mass * dx**2)
37             else
38                 e(N) = 0.0_dp
39             end if
40         end do
41     end subroutine initialize
42
43     function potential(x_vals) result(y)
44         real(dp), intent(in) :: x_vals
45         real(dp) :: y
46
47         y = lambda * (1 - exp(-a*x_vals))**2
48     end function potential
49 end module prob13_physics
50
51 program prob13_driver
52     use FDM_toolbox
53     use prob13_physics
54     implicit none
55
56     call initialize()
57     call tqli(N, d, e, z)
58     call sort_eigen(N, d, z)
59
60     write(*, '(A)') "-----"

```

```

61  write(*, '(A)') "Problem 13: Morse potential"
62  write(*, '(A)') "-----"
63  write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar
    = ", h_bar
64  write(*, '(A)') "Method: Finite Difference Method"
65  write(*, '(A)') "-----"
66  write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'
    ) " "

67
68  open(unit=10, file='data/prob13_energies.dat', status='replace')
69  do i = 1, 5
70      write(*, '(A, I2, A, G0.10)') "n = ", i-1, "      E = ", d(i)
71      write(10, *) d(i)
72  end do
73  close(10)
74
75  open(unit=10, file='data/prob13.dat', status='replace')
76  do i = 1, N
77      x = L_box_l + real(i-1, dp) * dx
78      write(10, *) x, potential(x), z(i,1), z(i,2), z(i,3), z(i, 4)
    , z(i, 5)
79  end do
80  close(10)
81  print *, "Wavefunctions written to prob13.dat"
82 end program prob13_driver

```

- **Output:**

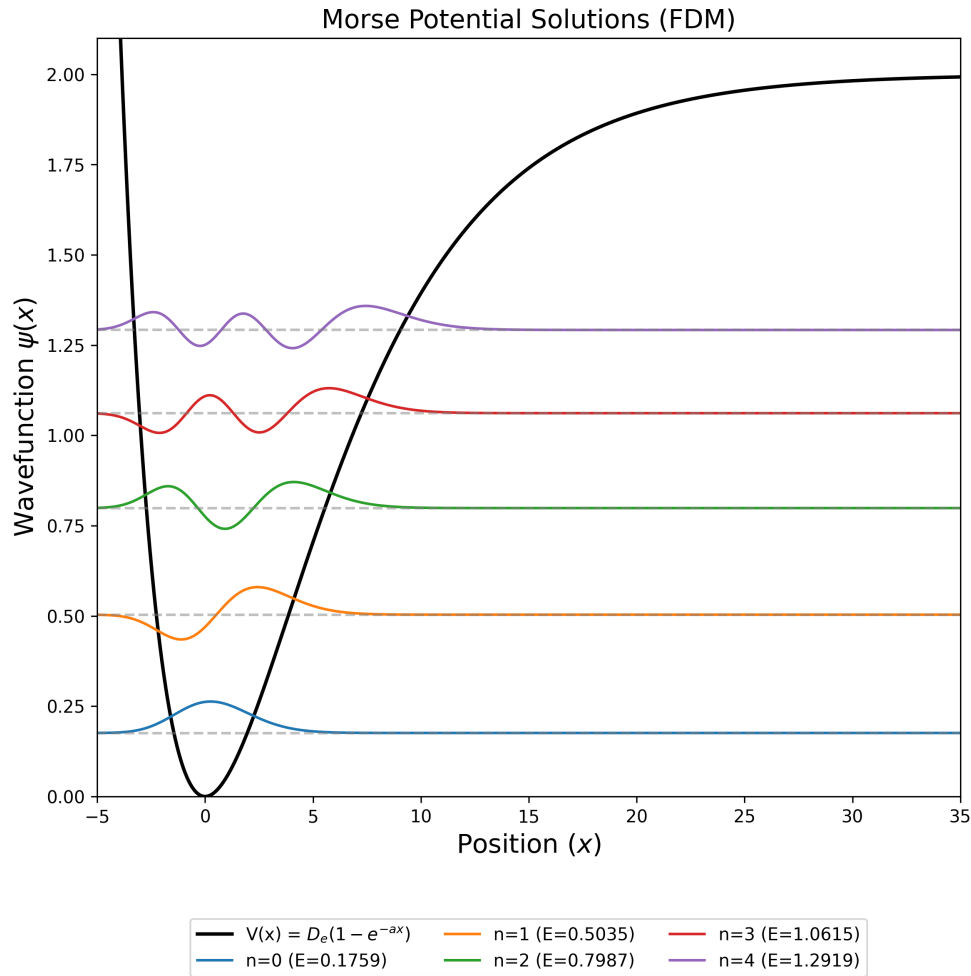
```

-----
Problem 13: Morse potential
-----
Unit used: m = 1.0, h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n = 0      E = 0.1759479931
n = 1      E = 0.5035415172
n = 2      E = 0.7987316204
n = 3      E = 1.061521094
n = 4      E = 1.291912169
Wavefunctions written to prob13.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 1.0 to prevent overlap between the energy levels.



- **Plot Analysis:** The numerical results highlight the anharmonicity of the Morse potential. Unlike the equidistant energy levels of the harmonic oscillator, the calculated energy gaps ( $E_{n+1} - E_n$ ) systematically decrease as the quantum number  $n$  increases, representing the weakening of the molecular bond at higher vibrational states. Visually, the wavefunctions exhibit broken symmetry. The left side of the probability distribution is sharply truncated by the steep repulsive wall, while the right side decays slowly, bulging outward into the wide dissociation region.

### 4.3 Problem 14: The Dirac Delta Potential

- **Description:** The attractive Dirac delta potential represents an idealized, infinitely deep, and infinitely narrow potential well. It is a critical theoretical test case because it analytically possesses exactly one bound state.
- **Potential:**  $V(x) = -\alpha\delta(x)$ . Because a computer grid cannot evaluate absolute infinity, the delta function is defined via the discrete grid spacing  $\Delta x$ . The potential is explicitly zero everywhere except at the exact coordinate origin ( $x = 0$ ), where it is approximated as:  $V(0) = -\frac{\alpha}{\Delta x}$ .
- **Code:**

```

1  !=====
2  ! File      : prob14_dirac_delta_potential.f90 (Additional Exercise)
3  ! Author    : Mohak Das
4  ! Course    : Computational Lab: Quantum Mechanics
5  ! Topic     : FDM approximation for Dirac Delta potential
6  ! Units     : m = h_bar = 1
7  ! Method    : Finite Difference Method
8  !=====
9
10
11 module prob14_physics
12     use constants_mod
13     implicit none
14     integer, parameter :: N = 2001
15     real, parameter :: alpha = 5.0_dp
16     real(dp) :: L_box = 6.0_dp
17     real(dp) :: x, dx
18     real(dp) :: d(N), e(N), z(N, N)
19     integer :: i
20
21 contains
22     subroutine initialize()
23         dx = (2 * L_box) / real(N - 1, dp)
24         z = 0.0_dp
25
26         do i = 1, N
27             z(i, i) = 1.0_dp
28         end do
29
30         do i = 1, N
31             x = - L_box + real(i - 1, dp) * dx
32             d(i) = ((h_bar**2) / (mass * dx**2)) + potential(x, dx)
33             if ( i < N ) then
34                 e(i) = - (h_bar**2) / (2 * mass * dx**2)
35             else
36                 e(N) = 0.0_dp
37             end if
38         end do
39     end subroutine initialize
40
41     function potential(x_vals, delta_x) result(y)
42         real(dp), intent(in) :: x_vals, delta_x
43         real(dp) :: y
44
45         if ( abs(x_vals) < 1.00e-10_dp ) then
46             y = -alpha/delta_x
47         else
48             y = 0.0_dp
49         end if
50     end function potential
51 end module prob14_physics
52
53 program prob14_driver
54     use FDM_toolbox
55     use prob14_physics
56     implicit none

```

```

57      call initialize()
58      call tqli(N, d, e, z)
59      call sort_eigen(N, d, z)
60
61
62      write(*, '(A)') "-----"
63
64      write(*, '(A)') "Problem 14: Dirac Delta potential"
65      write(*, '(A)') "-----"
66
67      write(*, '(A, G0.2, A, G0.2)') "Unit used: m = ", mass, ", h_bar
68      = ", h_bar
69      write(*, '(A)') "Method: Finite Difference Method"
70      write(*, '(A)') "-----"
71
72      write(*, '(A)') "The first five eigen energies: "; write(*, '(A)'
73      ) " "
74
75      open(unit=10, file='data/prob14_energies.dat', status='replace')
76      do i = 1, 3
77          write(*, '(A, I2, A, G0.10)') "n = ", i-1, "      E = ", d(i)
78          write(10, *) d(i)
79      end do
80      close(10)
81
82      open(unit=10, file='data/prob14.dat', status='replace')
83      do i = 1, N
84          x = - L_box + real(i-1, dp) * dx
85          write(10, *) x, potential(x, dx), z(i,1), z(i,2), z(i,3)
86      end do
87      close(10)
88      print *, "Wavefunctions written to prob14.dat"
89 end program prob14_driver

```

#### • Output:

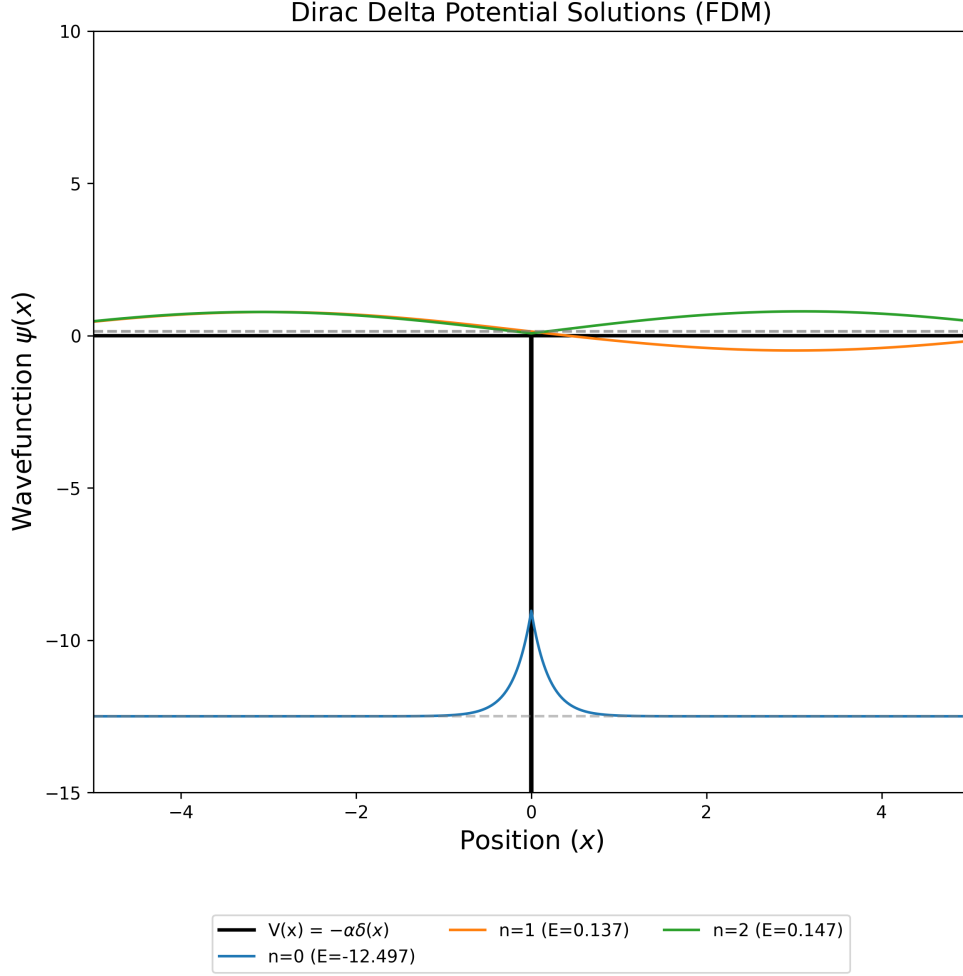
```

-----
Problem 14: Dirac Delta potential
-----
Unit used: m = 1.0, h_bar = 1.0
Method: Finite Difference Method
-----
The first five eigen energies:

n = 0      E = -12.49718876
n = 1      E = 0.1369386987
n = 2      E = 0.1465057862
Wavefunctions written to prob14.dat

```

- **Plot:** The wavefunctions were visualized using the standard scaling script (defined in Section 2.3) with a visual `scale_factor` of 20.0 to prevent overlap between the energy levels.



- **Plot Analysis:** The FDM solver successfully identified the single theoretical bound state ( $n = 0$ ,  $E_0 < 0$ ). The ground state wavefunction displays the characteristic sharp, discontinuous cusp at  $x = 0$  and decays exponentially outward. Fascinating numerical artifacts emerge for the excited states ( $n > 0$ ). Because these states possess positive energy ( $E > 0$ ), they are unbound scattering states. In the simulation, they are artificially trapped by the infinite walls of the computational domain  $L_{\text{box}}$ . Notably, the first excited state ( $n = 1$ ) possesses odd parity, enforcing a node at the origin ( $\psi(0) = 0$ ). Because the particle's probability density is strictly zero exactly where the delta potential exists, the particle completely ignores the well, resulting in a wavefunction identical to that of a free particle in an infinite square well.

## 5 Remarks

1. **Truncation Error:** The central difference approximation for the second derivative introduces an inherent  $O(\Delta x^2)$  truncation error. While increasing  $N$  reduces this error, it heavily impacts computational time.
2. **Artificial Boundary Conditions:** For extended potentials (like the finite well or double well), the computational domain  $L_{\text{box}}$  must be artificially truncated. If

$L_{\text{box}}$  is not sufficiently large, the enforcement of  $\psi = 0$  at the grid boundaries acts as an unintended infinite wall, artificially squeezing the wavefunction and raising the calculated energy eigenvalues.

3. **Numerical Precision Limits:** As observed in the double well potential, standard 64-bit floating-point precision ( $\sim 15$  decimal places) can be insufficient to resolve near-degenerate states. When the tunneling splitting drops below numerical noise, algebraic solvers fail to preserve the physical symmetry of the eigenvectors.